

CoNNA–Hardware accelerator for compressed convolutional neural networks

Rastislav J.R. Struharik^{a,*}, Bogdan Z. Vukobratović^b, Andrea M. Erdeljan^a,
Damjan M. Rakanović^a

^a University of Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad, 21000, Serbia

^b Kortiq GmbH, Gebrüder-Eicher-Ring 45, Forstern, Germany

ARTICLE INFO

Article history:

Received 6 January 2019

Revised 21 December 2019

Accepted 2 January 2020

Available online 11 January 2020

Keywords:

Machine learning

Convolutional neural network

CNN pruning

compressed CNN

Hardware acceleration

FPGA

ABSTRACT

In this paper, we propose a novel Convolutional Neural Network hardware accelerator called CoNNA, capable of accelerating pruned, quantized CNNs. In contrast to most existing solutions, CoNNA offers a complete solution to the compressed CNN acceleration, being able to accelerate all layer types commonly found in contemporary CNNs. CoNNA is designed as a coarse-grained reconfigurable architecture, which uses rapid, dynamic reconfiguration during CNN layer processing. The CoNNA architecture enables the on-the-fly selection of the CNN network that should be accelerated and also supports the acceleration of CNN networks with dynamic topology. Furthermore, by being able to directly process compressed feature and kernel maps, and skip all ineffectual computations during CNN layer processing, the CoNNA CNN accelerator is able to achieve higher CNN processing rates than some of the previously proposed solutions. The CoNNA architecture has been implemented using Xilinx ZynqUltrascale+ FPGA family and compared with seven previously proposed CNN hardware accelerators. Results of the experiments seem to indicate that the CoNNA architecture is up to 14.10, 6.05, 4.91, 2.67, 11.30, 3.08 and 3.58 times faster than previously proposed MIT's Eyeriss, NullHop, NVIDIA's Deep Learning Accelerator (NVDLA), NEURAghe, CNN_A1, fpgaConvNet, and Deephi's Aristotle CNN accelerators respectively, while using identical number of computing units and operating at the same clock frequency.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Deep learning [1], and particularly Deep Neural Networks (DNNs), are currently one of the most intensively and widely used machine learning predictive models. DNNs are not a new concept [2], but after recent breakthrough applications in the fields of image processing [3–5], and speech recognition [6], they have returned to the academic and industrial focus. Today, different types of DNNs are being employed in a wide range of applications, ranging from autonomous driving [7], medical [8], and even to playing complex games [9]. In many of these application domains, DNNs are now able to exceed human levels of performance. The exceptional performance of DNNs, and in particular Convolutional Neural Networks (CNNs) [3], predominantly arises from their ability to automatically extract high-level features from raw sensory data during the training phase, using a large amount of data, in order

to obtain an effective representation of input space. This approach is different from earlier attempts that have used manually crafted features or rules designed by experts. Because of this CNNs currently offer the best recognition quality versus alternative object recognition or image classification algorithms.

However, the superior accuracy of CNNs comes at a high cost because of their computational and storage complexity. State-of-the-art CNNs are described by hundreds of millions of parameters and require billions of computations in order to classify single input instance [3–5]. For example, one of the largest CNN networks, VGG-16 CNN [10], which operates on 224×224 input images, requires around 500 MB for storing network parameters and performing more than 30 billion floating-point operations in order to classify single input image. It is highly likely that future CNNs will be even larger, deeper, will process larger input instances, requiring even more computations per input instance, and will be used to perform more intricate classification tasks at faster speeds, ever-increasingly in real-time, within low-power operating conditions. Because of this, careful selection of appropriate computing platform for the implementation of CNN-based applications is of great importance.

* Corresponding author.

E-mail addresses: rasti@uns.ac.rs (R.J.R. Struharik), bogdan.vukobratovic@kortiq.com (B.Z. Vukobratović), andrea.erdeljan@uns.ac.rs (A.M. Erdeljan), rdamjan@uns.ac.rs (D.M. Rakanović).

Currently, there are two approaches to implementing CNN networks:

- 1 Using multicore processor-based hardware accelerators (CPUs or GPUs)
- 2 Using dedicated hardware accelerators (ASICs or FPGAs)

The first group of CPU/GPU based accelerators offers highly flexible and easy to develop solutions, in terms of supported CNN architectures, kernel parameters, nonlinear activation functions, pooling algorithms, and deep learning software frameworks (Caffe, TensorFlow, Keras, Matlab), but is inefficient when usage of available computing resources and power efficiency is considered. Standard CPU can perform between 10–100 GFLOP/s, with the power consumption that is usually around 100 Watts. Therefore, using CPUs in high-performance requirements, found in CNN cloud applications, or in low power requirements in edge/mobile applications is difficult. In contrast, GPUs can reach over 10 TOP/s of peak performance, but with power consumption reaching over 250 Watts [11], GPUs are good choices only for high-performance CNN cloud applications but are not suitable for the edge/mobile applications. NVIDIA is offering its Jetson TX2 family [12] based on GPUs with Pascal architecture as the edge solution. TX2 can reach the performance of up to 1 TOP/s but with the power consumption of over 10 Watts, which is still too high for most of the edge applications, which can require solutions with the power consumption of less than 1 Watt.

Dedicated hardware accelerators offer much higher utilization of computing resources, but usually lack flexibility, i.e. they support only a few different CNN architectures with smaller ranges of supported CNN architecture parameters, like kernel size, shape, etc. This is mostly the case because solutions from this group use highly optimized hardware structures for calculation of only certain pre-defined convolutional kernel configurations, which then cannot be reconfigured to efficiently calculate other convolutional kernel configurations. However, because of the higher utilization of available computing resources, accelerators from this group tend to be more power-efficient than the processor-based accelerators.

While general-purpose compute engines, especially GPUs, have been the mainstay for much of contemporary CNN processing, increasingly there is a growing interest in providing more CNN implementations based on FPGAs. This shift is occurring mainly because of two reasons. Improvements in FPGA technology recently demonstrated FPGA performance which comes very close to GPU performance, with the reported performance of 9.2 TOP/s for FPGA [13]. Second, recent trends in CNN architecture development increasingly exploit the sparsity of CNN networks and the use of extreme compact data types to represent data that is being processed. These trends strongly favor FPGA devices, which are designed to easily handle irregular parallelism, which is present when working with sparse CNNs, and custom data types. As a result, next-generation CNN accelerators are expected to deliver up to x5.4 better computational throughput than GPUs [14].

The field of custom CNN hardware accelerators has been in the focus of the academic community in recent years, generating more than ninety different solutions. However, the majority of these solutions are concerned with acceleration of only one particular layer type from the CNN, typically the convolutional layer, because of its high computational demand, [15–19]. A significant number of architectures have been proposed for the acceleration of complete CNNs [20–30]. Furthermore, most of the proposed solutions are able to accelerate only uncompressed CNNs [15–30], with only several examples being able to process compressed CNNs to some degree [31–36]. In addition, almost all proposed solutions are not highly configurable, severely limiting the flexibility of supported CNNs, in terms of supported layer types, kernel receptive field sizes, horizontal and vertical kernel stride values, pooling

types, types of non-linear activation functions, etc. This is one of the main drawbacks of the majority of previously proposed custom hardware solutions, which puts them at a great disadvantage when compared with CPU/GPU implementations.

Both groups of accelerators (CPU/GPU and custom) tend to use a sequential approach when processing the CNN network. CNN network is processed layer by layer, storing intermediate values in the external, off-chip DRAM memory. Usually, no parallelization of layer processing is employed. Please notice, that this frequent movement of data between the accelerator and DRAM memory is one of the most power-consuming operations, and accounts for the majority of power consumption of the complete accelerator system.

Parallelization in both groups of accelerators mostly comes from the parallel evaluation of individual convolution operations, located within convolutional layers, usually using a 2D array of processing elements. Multiplications and additions involved in the convolution calculation are executed in parallel, on dedicated multiplier and adder modules located within each processing element. Although this is an efficient way of achieving higher performance numbers, it has one significant drawback. Mapping of convolution operations of different sizes (with different sizes and shapes of convolutional kernels and with different values of kernel stride values) to these 2D array structures can be very difficult, and even when it is possible typically results in inefficient usage of available multipliers and adders, therefore significantly decreasing the efficiency of an accelerator.

In this paper, we present a novel, coarse-grained reconfigurable, compressed CNN hardware accelerator, named CoNNA, which aims to overcome these difficulties. The CoNNA architecture is based on a different way of parallelizing CNN operations. It sequentially computes individual convolution operation, keeping it folded, using a single processing unit to calculate all multiply-accumulate operations contained within one convolution, but employs a number of these processing units to compute a number of convolutions in parallel. This parallelizing approach can easily accommodate convolutional kernels of different sizes, shapes and stride values, without significant loss on the computational efficiency.

Furthermore, CoNNA is designed to be able to directly operate on compressed CNN networks and compressed feature maps, which results in a significant increase in CNN processing performance. The CoNNA architecture is highly configurable, enabling various types of CNN families (VGG, Inception, ResNet, MobileNet, NASNet, etc.) to be implemented efficiently. It is also capable of accelerating complete compressed CNNs, supporting all major layers found in contemporary CNNs, like convolutional, depthwise convolutional, pooling, adding and fully-connected layers.

The CoNNA architecture is not the first architecture that takes benefit from processing sparse, compressed CNNs. Some of the most notable previously proposed compressed CNN accelerators are Cnvlutin [31], NullHop [32], SparseNN [33], Cambricon-x [34], EIE [35], and Scnn [36]. Although not being the first compressed CNN accelerator, CoNNA still has some important advantages over all these previously proposed architectures.

Cnvlutin architecture [31], proposed by Albericio et al. benefits from the sparsity of input feature maps but not the sparsity of weights. It is closely based on the well-known dense CNN accelerator DaDianNao [15], where the authors of Cnvlutin have re-designed the DaDianNao's NFU module and created a CNV module, capable of efficient detection and skipping of zeros present in input feature maps. The CoNNA architecture can also skip input feature maps zeros, but it can also skip any zeros that are present in the convolutional kernels, or fully-connected weights, which should result in more efficient CNN processing when compared to Cnvlutin. Furthermore, Cnvlutin is designed to perform complete CNN processing storing all intermediate data in the on-chip memory. This

is a power-efficient solution but requires large on-chip memories, of the order of tens of megabytes, and cannot support arbitrarily large CNN networks. On the other hand, CoNNA stores intermediate feature maps in the external DRAM memory, but the power consumption is reduced significantly because CoNNA uses an on-chip feature map cache to minimize the amount of data movement between the accelerator and the external memory. However, by using external memory to store all CNN related data, CoNNA can provide acceleration to arbitrarily sized CNN networks.

NullHop architecture [32], proposed by Aimar et al. is similar to Cnvlutin since it also takes benefit from skipping any zeros that are present in the input feature maps but cannot skip zeros in the CNN weights. Since the CoNNA architecture can effectively skip all zeros, present either in the input feature maps or in the CNN weights, it should offer a more efficient acceleration of CNNs. Similar to CoNNA, NullHop stores both CNN weights and compressed intermediate feature maps in external memory, but NullHop doesn't implement any input feature map caching, which CoNNA does, to reduce the required data movement between the accelerator and external memory when processing convolutional and pooling layers.

SparseNN [33] and Cambricon-x [34] architectures take the opposite approach than one taken in Cnvlutin and NullHop architectures, by skipping zeros that are present in the CNN weights but are not able to skip zeros that are present in the input feature maps. By being able to skip zeros that are present both in input feature maps and CNN weights the CoNNA architecture should offer more efficient CNN acceleration than Cambricon-x and SparseNN architectures.

Similar to CoNNA, EIE architecture [35] can skip zeros located both in input feature maps and CNN weight. But, EIE is able to do this only for fully-connected layers, and not for convolutional layers. Therefore, EIE architecture cannot be used to process convolutional, pooling or adding layers from CNNs, which makes EIE an interesting solution for the acceleration of fully-connected CNN layers, but not the complete compressed CNN accelerator. On the other hand, the CoNNA architecture is designed to be capable of accelerating all standard CNN layer types, including convolutional, depthwise convolutional, pooling, adding and fully-connected layers. Furthermore, during the acceleration of convolutional and fully-connected layers, CoNNA is able to skip all zeros that are present in the input feature maps and convolutional coefficient and fully-connected weight maps. This makes CoNNA a universal pruned CNN accelerator, as opposed to the EIE architecture.

Scnn architecture [36] can also skip all zeros located both in input feature maps and CNN weights, similar to EIE and CoNNA architectures, but it can process only convolutional layer type. So the Scnn architecture is to a degree complementary to the EIE architecture. Both architectures can skip all zeros but in different layer types. Once more, CoNNA is able to skip all zeros both in convolutional and fully-connected layers, and opposed to EIE and Scnn architecture offers a solution to hardware acceleration of complete pruned CNNs.

The rest of the paper is organized as follows. Section 2 presents a brief introduction to CNNs, with the emphasis on CNN compression, since CoNNA takes great benefit from CNN compression. Section 3 presents details of the proposed CoNNA architecture, describing all of its major components. Section 4 contains the results of the experiments aimed at comparing the performance of the CoNNA architecture with some of the previously proposed CNN hardware acceleration solutions, and also presents a detailed discussion about the impact of the irregular distribution of non-zero input feature map and CNN weight values on the CoNNA's CNN processing efficiency. Section 5 holds final remarks and conclusions.

2. Compressed convolutional neural networks

Convolutional Neural Network [37] is a type of feed-forward artificial neural network in which the connectivity pattern between the neurons is inspired by the neural connectivity found in the animal visual cortex. Individual neurons from the visual cortex respond to stimuli only from a restricted region of space, known as the receptive field. Receptive fields of neighboring neurons partially overlap, spanning the entire visual field. Previously it was shown that the response of an individual neuron to the stimuli within its receptive field can be approximated mathematically by a convolution operation, which is extensively used in CNNs.

CNN architecture is formed by stacking together layers of differentiable functions, which gradually transform input instance into an appropriate output response (e.g. holding the class scores). A number of different layers types are commonly used when building a CNN: convolutional layer, depthwise convolutional layer, pooling layer, non-linear activation layer, adding layer, concatenation layer, and fully-connected layer.

Before proceeding, let us introduce some notations. By an input feature map (IFM) we will assume a 3D set of values that make the input volume of the current CNN layer. The input values volume of the first CNN layer is usually not called an input feature map; rather it is called an input instance. Similarly, an output feature map (OFM) designates the 3D set of activation values of every neuron present in the current CNN layer. The output volume of the last CNN layer is usually not called the output feature map; rather it is called the classification vector.

A 3D region of $N \times M \times D$ points within the input feature map that is directly connected to one neuron from the current CNN layer will be designated as the input feature map bundle. In most CNN architectures values for the horizontal and vertical size of the input feature map bundle are equal, $N = M$. For example, the size of the input feature map bundle of the 4th convolutional layer from the VGG-16 CNN network [10] is a 3D region of $3 \times 3 \times 128$ points. Similarly, the size of the input feature map bundle for the 2nd pooling layer from the VGG-16 network is a 3D region of $2 \times 2 \times 1$ points.

A 3D region of $1 \times 1 \times D$ points within the feature map bundle will be designated as the feature map stick. If the feature map is the input feature map, then the feature map stick will be called the input feature map stick. Accordingly, if the feature map is the output feature map, then the feature map stick will be called the output feature map stick. Every input feature map bundle is composed of a number of input feature map sticks. For example, the input feature map bundle of the 4th convolutional layer from the VGG CNN is composed of 9 input feature map sticks, each being a 3D region of $1 \times 1 \times 128$ points.

Fig. 1 illustrates the definitions of the feature map stick and the feature map bundle, in the case when the feature map bundle is composed of 9 feature map sticks.

CNNs are both computationally and memory demanding, which makes their usage difficult, especially in embedded applications. For example, VGG-16 CNN [10] has more than 138 million network parameters. Using 32-bit floating number representation, approximately 552 MB of memory space must be available only for storing all required network parameters. Furthermore, during layer processing, CNN uses an input feature map, which is either the input image itself, or the output of the previous CNN layer, and produces an output feature map (OFM). The sizes of input and output feature maps depend on the characteristics of CNN layers. For example, the largest input and output feature maps in the case of the VGG-16 CNN are more than 12 MB large each, when 32-bit floating-point representation is used. Compared to the size of the memory required to store VGG-16 CNN parameters, the size of the required memory for storing all intermediate feature maps is only

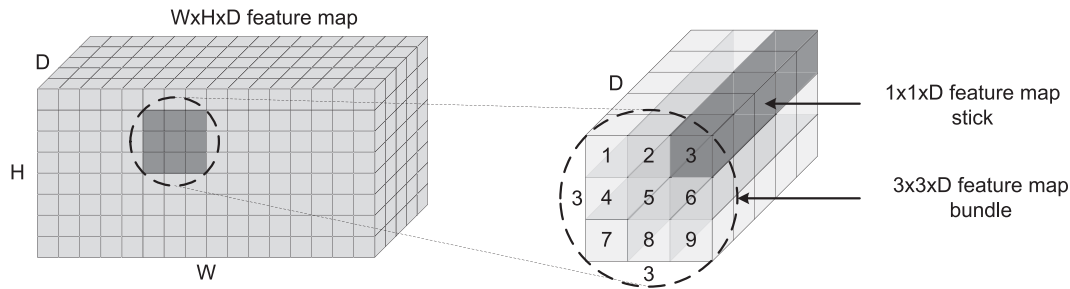


Fig. 1. Illustration of definitions of feature map bundle and feature map stick.

around 6% of the required network parameters memory size. However, the total required feature map data movement size to process one input image reaches around 120 MB. If we analyze the number of required computations in order to classify one input image, in the case of VGG-16 CNN it reaches over 15 G of MAC operations, or equivalently, over 30 GOPs. For example, if VGG-16 CNN is deployed in a system that is processing real-time video data stream, with a speed of 25 frames-per-second, the required compute power of CNN implementation system reaches 750 GOP/sec with required data throughput of at least 16.8 GB/sec. This makes deployment of CNNs in embedded applications very challenging, especially when latency, throughput and/or power consumption is of interest.

One way of solving these issues is to reduce the memory size required to store network parameters and intermediate feature maps, by using different CNN compression methods. Generally, all CNN compression methods can be divided into two groups, depending on the target data: methods that compress CNN network parameters, and methods that compress feature map data.

CNN pruning [38–45] is the major technique used to compress CNN network parameters. CNN pruning can be beneficial because of a number of reasons:

1. By removing redundant weights memory footprint of the target CNN network is reduced, allowing it to be used in memory-constrained applications. Since pruned weights are actually set to zero, resulting pruned CNN is a sparse structure, containing many zeros within its convolutional kernel and fully-connected weight maps. Some of the data compression techniques could be used to compress this sparse representation of CNN, which will then require a smaller amount of memory space for its storage.
2. Trainable CNN weights are dominantly located within convolutional and fully-connected layers and are always used in multiplicative operations. If some of these weights are set to zero, because they are redundant, product terms that involve these zero-valued weights actually don't have to be computed, because their value will also be equal to zero. If we could detect these zero-outcome product terms and skip them, the processing time of convolutional or fully-connected layers could be reduced, resulting in shorter input instance processing time. The process of skipping zero-valued product terms is also known as the "Zero Skipping" process, and the CoNNA accelerator is designed to use it during CNN processing. Speeding-up instance processing by zero skipping can be beneficial because it can either allow us to process input instances at a faster rate, resulting in higher throughput, or alternatively save power because we could run CNN accelerator at a slower clock frequency and still be able to finish with instance processing on time.
3. Finally, since the compressed sparse representation of CNN network is significantly smaller in size when compared with the original, un-pruned, dense representation of CNN, a significant amount of energy can be saved during movement of CNN pa-

rameters, especially if CNN parameters are stored in external, off-chip memory, which is usually the case.

A critical step during the CNN network pruning process requires identifying kernel coefficients or fully-connected weights that are redundant and can be removed. Since this can be done in numerous ways, a large number of different pruning algorithms have been proposed in the open literature [38–45], with the new ones continuously being proposed. However, all CNN pruning algorithms can be broadly divided into two large categories:

1. Coarse-grained pruning algorithms – these algorithms remove complete convolutional kernels (entire filters) from a convolutional layer. Some of the previously proposed coarse-grained pruning algorithms are [38–41].
2. Fine-grained pruning algorithms – these algorithms remove individual kernel coefficients from selected filter within convolutional layers, leaving the original number of filters unchanged, or remove individual weights from fully-connected layers. Some of the previously proposed fine-grained pruning algorithms are [42–45].

One widely used CNN network parameter compression algorithm is the "Deep Compression" algorithm, proposed by Song Han, et al. in [44]. "Deep Compression" algorithm uses a three-stage pipeline to reduce the storage size required by the CNN network, in a manner that preserves original accuracy. First, the original CNN network is pruned by removing all redundant kernel coefficients and fully-connected weights, keeping only the most informative ones, using fine-grained pruning algorithm. Next, remaining connection weights are quantized so that multiple connections share the same weight, thus only the codebooks (effective weights) and the indices need to be stored. Finally, Huffman encoding is applied to take advantage of the biased distribution of effective weights. After performing all three steps of the "Deep Compression" algorithm, the size of memory required for storing all CNN parameters can be reduced significantly. For example, in the case of VGG-16 CNN, the required memory size is reduced from 552 MB to only 5.5 MB.

Please notice that pruning and weight quantization affect the overall accuracy of the CNN network, but as shown in [44], significant pruning and quantization can be applied to standard CNNs, trained on standard datasets, without degrading initial, unpruned, CNN network accuracy. In this paper, we also use fine-grained pruning to sparsify CNN network parameters before the sparse CNN network is run on the CoNNA CNN accelerator. However, we don't cluster remaining weights into codebooks, nor do we use Huffman encoding. However, we do perform quantization of unpruned weights into a 16-bit fixed-point representation to further reduce the memory footprint and enable more efficient FPGA implementation.

In a fine-grained pruning approach, each convolutional kernel is pruned individually. Usually, a specified number of kernel coefficients are removed from each kernel, to create a sparse convo-

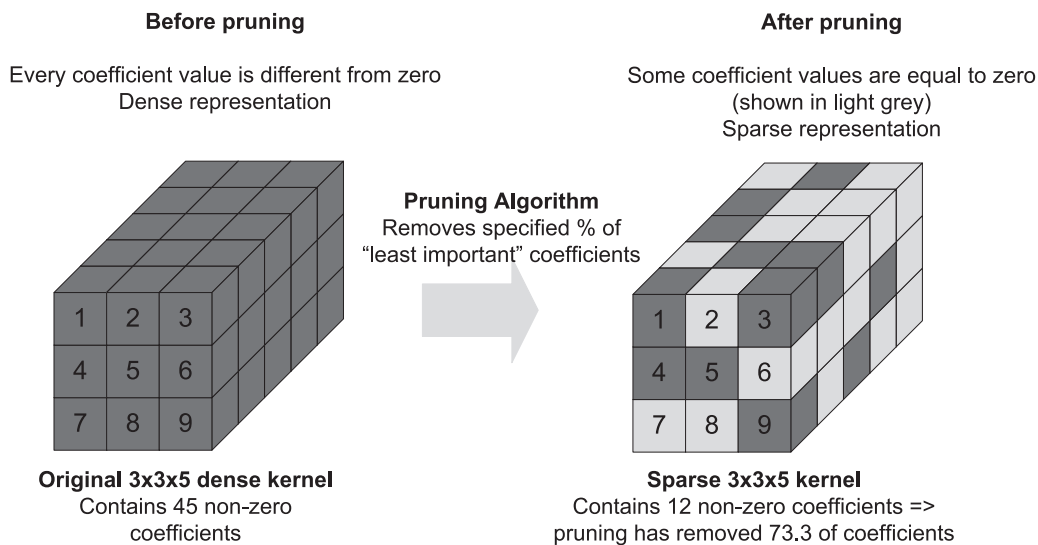


Fig. 2. Fine-grained CNN pruning principle of operation.

lutional kernel. For example, Fig. 2 illustrates the process of fine-grained pruning of a $3 \times 3 \times 5$ convolutional kernel.

We start with the dense representation of the convolutional kernel, shown on the left of Fig. 2. Dense kernel coefficients are obtained during the training process of the selected CNN network. During fine-grained pruning, a specified number of kernel coefficients are removed, effectively setting them to zero. In the example from Fig. 2, a total of 33 kernel coefficients are removed from dense $3 \times 3 \times 5$ kernel, to obtain a sparse $3 \times 3 \times 5$ kernel, shown on the right of Fig. 2. In this sparse representation of $3 \times 3 \times 5$ kernel, only 12 coefficients have non-zero values, while the remaining 33 coefficients are set to zero.

Criteria upon which kernel coefficients are selected for pruning can be numerous. One commonly used approach is to prune kernel coefficients with the smallest absolute values. In this approach, selected kernel coefficients are first sorted in ascending order of their absolute value, and the specified number of smallest valued coefficients is then set to zero. This process is repeated for all convolutional kernels from all convolutional layers of the target CNN network, with possibly different pruning ratios for each individual convolutional layer. Weight values from fully-connected layers can also be pruned using a similar approach in order to create a sparse weight map for every fully-connected layer.

After CNN network pruning has been performed the result is a sparse representation of selected CNN network, containing a significant number of zero-valued convolutional coefficients or fully-connected weight values. Usually, especially when high pruning ratios are employed, the resulting pruned CNN network has significantly lower accuracy, compared with the original dense, unpruned CNN network. Therefore, an additional retraining process of the pruned CNN network is necessary, in order to regain the accuracy of the unpruned CNN network. Depending on how severe the CNN pruning process has been, retraining of pruned CNN network can restore complete or only a fraction of original unpruned CNN network accuracy.

Although CNN network pruning can be performed incrementally, layer by layer, most common approach is to prune all applicable CNN layers at the same time, create one sparse representation of complete CNN network and then retrain it, in order to regain as much of original accuracy. Please notice that the presented pruning algorithm was used in this paper to prune the CNN networks prior to their acceleration by the CoNNA CNN accelerator.

The amount of pruning that can be used with given CNN network depends on a number of factors: CNN network architecture,

underlying problem that CNN network is being trained to solve, pruning algorithm that is being used, distribution of pruning ratios over layers from CNN network selected for pruning, optimization algorithm used to train CNN network, etc. Achieving the highest pruning ratios for the selected layers from the CNN network can be a very time-consuming, trial and error, process.

Please notice that pruned, sparse representations of CNN networks by themselves don't reduce the memory size required for storing network parameters. They also don't reduce the amount of data that will be moved between the CNN accelerator and operating memory, nor do they reduce the time required to process the input images. Pruning only introduces a certain amount of zero-valued parameters within the CNN network description, but the total number of network parameters remains the same as in the original unpruned CNN network.

Therefore, the amount of memory required to store pruned CNN network parameters will be identical with the amount required to store parameters of the unpruned CNN network, the only difference being that a certain amount of these parameters will have zero value in the case of pruned CNN network. Similarly, the amount of data movement between CNN accelerator and the operating memory, where the CNN network parameters are stored, will be identical for unpruned and pruned CNN network representations, the difference once more being the fact that in the case of pruned CNN network many of parameters that will be transferred to the accelerator will be zero-valued.

However, since pruned CNN network representation contains a significant number of zero-valued parameters, lossless data compression algorithms can be used to reduce the amount of data required to store pruned CNN network representation. The CoNNA architecture uses a variation of standard Zero Run-Length (ZRL) encoding compression algorithm [19] to compress sparse pruned CNN representations. Using ZRL compression a significant reduction in the memory size required to store pruned CNN network representation and the total data transfer size required to transfer pruned CNN network representation to the CNN accelerator, can be achieved. For more details about the ZRL compression algorithm used within the CoNNA architecture and the amount of data storage and transfer size reduction that is achievable, please refer to Horowitz [46].

Please also notice that, compared to the unpruned CNN network, having a pruned CNN network doesn't reduce the number of MAC operations that need to be performed in order to process one input image. The only difference will be that in the

case of a pruned CNN network, a number of these MAC operations will result in zero value since we will be multiplying input feature map points with zero-valued convolutional coefficients or fully-connected weight values. Reduction in instance processing time could be possible if the CNN accelerator could somehow “detect” and “skip” these zero-valued MAC operations since they don’t change the final outcome of convolution or weighted sum calculations. The CoNNA architecture was designed specifically to exploit this optimization opportunity.

Reduction in memory size required for storing CNN network representation, reduction in required data transfer size and reduction in instance processing time can have beneficial effects in lowering the cost and increasing the energy efficiency of CNN network processing, which could be of special interest, especially in embedded, edge-computing applications.

To further improve energy and compute efficiency of CNN network processing, statistics of feature map data being processed by CNN network can also be explored to reduce the number of required memory accesses, using feature map compression technique. Efficient feature map compression is possible because of the heavy use of the ReLU activation function within CNN layers. The ReLU activation function introduces a significant number of zeros in the intermediate feature maps, by rectifying all negative output feature map values to zero. The number of zeros in feature maps depends on the information content of input data being fed to CNN, CNN architecture that is being used to process input data, as well as the underlying problem that the CNN network is trained to solve. Nevertheless, it tends to increase as we move deeper into the CNN network. For example, in VGG-16 CNN almost 48% of the IFM values of the CONV1_2 layer are zeros on average, and this value goes up to 88% for the CONV5_3 layer, Chen et al. [19].

Zero Run-Length encoding [19] has again been proposed to efficiently exploit this phenomenon and compress zero values found in the feature maps, by replacing consecutive runs of zeros with the single run-length value. Using ZRL encoding only adds 5%–10% overhead to the theoretical entropy limit [19]. For example, in the case of VGG-16 CNN, using ZRL encoding results in the reduction of required feature map transfer size per input image, from 60 MB in the uncompressed case, to only 30 MB in compressed case, when 16-bit fixed-point representation of feature map data is used [19].

In the typical usage of feature map compression, all feature maps, except input image, are stored in compressed format in operating memory. Accelerator reads encoded IFMs from operating memory, decompresses them using an appropriate decoder, and uses a decompressed data stream within the accelerator to compute output feature map. Computed output feature maps are optionally processed by the ReLU module, compressed by the ZRL encoder, and transmitted to external memory. This saves both space and R/W bandwidth of the external memory. For example, this approach is used in MIT’s Eyeriss accelerator [19].

However, please notice that a further improvement in the CNN processing time is possible if all MAC operations where IFM points are zero-valued are “skipped” since they once more don’t change the final outcome of convolution or weighted sum calculations, similar to “skipping” MAC operations with zero-valued convolutional coefficients or fully-connected weight values, described previously.

In principle, having a significant number of zero values in input feature maps, as well as in convolutional kernel and fully-connected weight maps can be used to improve CNN instance processing efficiency by using one of three possible “zero-skipping” techniques:

1. IFM “zero-skipping” - in this approach all MAC operations where IFM point value is zero are skipped during the process of convolutional and weighted sum calculation. For example, CNN accelerators proposed in [31,32] use this optimization technique to increase CNN processing efficiency.
2. Weights “zero-skipping” - in this approach all MAC operations where convolutional coefficient or fully-connected weight value is zero are skipped during the process of convolutional and weighted sum calculation. For example, CNN accelerators proposed in [33,34] use this optimization technique to increase CNN processing efficiency.
3. “All zero-skipping” - in this approach all MAC operations where either IFM point or convolutional coefficient/fully-connected weight value is zero are skipped during the process of convolutional and weighted sum calculation. For example, CNN accelerators proposed in [35,36] use this optimization technique to increase CNN processing efficiency.

Each of the three presented “zero-skipping” techniques lead to an improvement in the CNN instance processing time, but it is clear that the “All zero-skipping” technique leads to the most significant improvement, although it is the most difficult to implement. Proposed CoNNA architecture is designed specifically to implement this third, “All zero-skipping” optimization technique. Please notice that the majority of the previously proposed CNN accelerators [15–30] have completely ignored this “zero-skipping” optimization opportunity.

3. CoNNA CNN accelerator details

The CoNNA CNN accelerator is specifically designed to exploit the “All zero-skipping” optimization opportunity, described in Section 2. The CoNNA architecture is designed to accelerate compressed pruned CNNs, created using some of existing fine-grained CNN pruning algorithms, for example [38–45]. As the result of the application of CNN pruning procedure, CNN kernel and weight maps will be sparse, since a certain number of weights will be removed from the CNN during pruning step. Similarly, because of ReLU activation function use, a significant number of intermediate feature map values will be zero also. CoNNA is able to detect on-the-fly all product terms that will result in the zero-valued outcome and skip their calculation, therefore reducing layer processing time significantly. This is done when convolutional, pooling and fully-connected layers are being processed by the CoNNA. Furthermore, CoNNA is able to process feature and kernel maps data in compressed form, removing the need for decompression step, thus further shortening processing time and reducing the size of on-chip memories used for storing IFM and KM data, which is a significant improvement compared to existing solutions [19].

3.1. Principle of operation of the CoNNA CNN accelerator

More than 90% of the operations in CNN processing involve convolutions [3,10,51,52]. Therefore, optimizing the execution of convolutional layers has an overwhelming impact on the overall efficiency and performance of any CNN accelerator. The process of transforming an input feature map into an output feature map by a convolutional layer can be described by Algorithm 1. Please note that, for the reason of simplicity, in presented pseudo-code adding of bias term for each convolutional kernel and optional IFM padding (necessary if we want to keep the horizontal and vertical size of OFM equal to that of IFM) were omitted.

A convolutional layer takes as an input a feature map which can be represented as a 3D tensor of dimension $IFM_Width \times IFM_Height \times IFM_Depth$. The convolutional layer then transforms this input feature map, using $Kernel_Num$ different convolutional kernels, into an output feature map of dimension $OFM_Width \times OFM_Height \times Kernel_Num$. Each convolutional kernel K is a 3D tensor of dimension $Kernel_Width \times Kernel_Height \times Kernel_Depth$,

Algorithm 1 Pseudo-code of generic convolutional layer processing algorithm.

```

L1: for (y = 0; y < OFM_Height; y++)
L2:   for (x = 0; x < OFM_Width; x++)
L3:     for (kn = 0; kn < Kernel_Num; kn++)
L4:       for (kd = 0; kd < Kernel_Depth; kd++)
L5:         for (kw = 0; kw < Kernel_Width; kw++)
L6:           for (kh = 0; kh < Kernel_Height; kh++)
             OFM[x][y][kn] +=
             IFM[x*Sh+kw][y*Sv+kh][kd]*KM[kn][kw][kh][kd];

```

where *Kernel_Depth* parameter must be equal to the *IFM_Depth* parameter. Since *Kernel_Width* and *Kernel_Height* parameters are usually much smaller than *IFM_Width* and *IFM_Height* parameters, each convolutional kernel is actually convolved many times, each time with different IFM bundle, using notation introduced in Fig. 1. The way how kernels traverse IFM is controlled by two additional parameters, horizontal and vertical kernel stride values, *Sh* and *Sv*. Input feature map horizontal and vertical dimensions, stride parameters, together with horizontal and vertical padding values determine the horizontal and vertical dimensions of resulting output feature map, *OFM_Width*, and *OFM_Height* respectively. The depth of the resulting output feature map is unrelated to the depth of the input feature map and is purely determined by the number of different convolutional kernels that are defined within the current convolutional layer, *Kernel_Num*.

As can be seen from the pseudo-code of Algorithm 1, the process of computing the output from a convolutional layer requires going through six nested loops. Please notice that there can be an additional 7th, outermost loop if batch processing is used, but for simplicity reasons, it is not present in Algorithm 1. At the core of these nested loops are the MAC operations, where selected IFM and KM points are multiplied and then added to the running sum of appropriate OFM point. This six nested loops algorithmic description of convolutional layer operation creates a rather large design space of possible computing architectures, each of them implementing a different type of parallelism, sequencing computations and partitioning large IFM and KM data into smaller blocks that can then more easily fit into smaller on-chip memories. It is worth noting that each of the previously proposed CNN hardware accelerator architectures is actually representing one of these design space points, CoNNA architecture included.

The CoNNA architecture is based on the idea of sequential computation of individual convolutions from a convolutional layer, using a single Processing Block (PB) unit for each convolution that needs to be computed, and employing a number of PB units to compute several different convolutions in parallel. Algorithm 2 presents the pseudo-code of the convolutional layer processing algorithm implemented in the CoNNA architecture.

Algorithm 2

Pseudo-code of convolutional layer processing algorithm, Implemented by the CoNNA architecture.

```

L1: for (y = 0; y < OFM_Height; y++)
L2:   for (x = 0; x < OFM_Width; x++)
L3:     for (kn = 0; kn < Kernel_Num; kn+=Num_PB)
L5:       for (kw = 0; kw < Kernel_Width; kw++)
L6:         for (kh = 0; kh < Kernel_Height; kh++)
L4:           for (kd = 0; kd < Kernel_Depth; kd++)
             {OFM[x][y][kn] +=
             IFM[x*Sh+kw][y*Sv+kh][kd]*KM[kn][kw][kh][kd];
             OFM[x][y][kn+1] +=
             IFM[x*Sh+kw][y*Sv+kh][kd]*KM[kn+1][kw][kh][kd];...
             OFM[x][y][kn+Num_PB-1] +=
             IFM[x*Sh+kw][y*Sv+kh][kd]*KM[kn+Num_PB-1][kw][kh][kd];}

```

Comparing Algorithm 2 with Algorithm 1 it can be seen that two modifications have been made. The first modification makes loop L4 the innermost loop, which means that during the calculation

of individual convolutions CoNNA architecture processes IFM bundles in the “Stick First” manner. Each IFM bundle that should be convolved is divided into a number of IFM sticks, using notation from Fig. 1, and then each individual IFM stick is multiplied with the corresponding convolutional coefficients from appropriate convolutional kernel stick. Furthermore, each individual convolution is calculated sequentially, meaning that loops L4-L6 are kept rolled. This is in sharp contrast with most of the previously proposed CNN accelerators that try to increase processing speed by partially or totally unrolling some or all of loops L4-L6.

Sequential approach of calculating individual convolutions employed by the CoNNA architecture is highly flexible regarding the parameters of convolutions (kernel size, horizontal and vertical stride, kernel shape, etc.), resulting in highly-configurable accelerator, almost to the level of the accelerators based on CPUs and GPUs, while still having high utilization of PB units, equal or even higher than the MAC utilization of the accelerators based on the custom hardware solutions. This high flexibility is in sharp contrast with most of previously proposed CNN accelerators, which usually support only certain kernel configurations, allowing for example only 3×3 or 5×5 kernels with the stride of 1. Even if they do support arbitrary kernel size and stride values, their efficiency drops significantly when these parameters are set to some non-standard values.

Since individual convolutions are calculated sequentially, one MAC operation at a time, CoNNA architecture can easily support convolutional kernels of any size, with different horizontal and vertical sizes, with different horizontal and vertical strides, without any degradation of the processing efficiency. Furthermore, the shape of the convolutional kernel doesn't have to be squared, it can be rectangular also, and it can even be triangular, oval, or any other shape for that matter, including asymmetric shapes also. Since each convolutional kernel is processed sequentially, one coefficient at a time, its actual shape and size are of no relevance to the CoNNA architecture, and it also doesn't affect the efficiency of computing individual convolution.

The second modification of Algorithm 1 that is present in Algorithm 2 is the partial unrolling of loop L3, with the partial unrolling factor equalling the number of available Processing Block modules, *Num_PB*. This parameter is actually one of the configuration parameters of the CoNNA architecture and can be specified at compile time to generate an instance of the CoNNA architecture with a specified number of Processing Blocks. This configurability enables the scalability of the CoNNA architecture in terms of achievable instance processing performance. As can be seen from Algorithm 2, CoNNA uses existing Processing Block modules to concurrently compute a number of individual convolutions. All these convolutions involve the same IFM bundle but use different convolutional kernels.

Partial unrolling of loop L3 actually is one way of speeding-up the CNN processing used within the CoNNA architecture, by parallelizing the process of computation of individual convolutions. Another way of speeding-up CNN processing used in CoNNA is skipping ineffectual MAC operations, by skipping all product terms that result in the zero-valued outcome. As previously stated, the CoNNA architecture implements “All zero-skipping” technique, presented in Section 2, to skip all unnecessary multiplications, which will result in zero-valued outcome during convolutional, pooling and fully-connected layer processing, so the actual convolutional layer processing algorithm that is implemented in the CoNNA architecture is slightly different from Algorithm 2. Algorithm 3 presents the pseudo-code of the final convolutional layer processing algorithm, using the “All zero-skipping” technique, which is actually implemented in the CoNNA architecture.

As opposed to Algorithms 1–3 operates on a compressed input feature map, represented by *CIFM* tensor, and compressed convolu-

tional kernels, represented by CKM tensor. Compressed input feature map is the result of compressing zero-valued activations that are present due to the use of the ReLU activation function, while compressed convolutional kernels are the result of the CNN pruning process, described in Section 2. These compressed structures contain only non-zero input feature map and kernel values, together with the information about their positions in the original, uncompressed structures.

Algorithm 3 implements the “All zero-skipping” technique within the W1 loop, which replaces loops L4-L6 from Algorithms 1 and 2. W1 loop is actually a *while* loop that is active until all Processing Blocks have finished computing all non-zero product terms from their associated convolution. When all PBs complete their convolution calculation tasks, *PBs_Busy()* function will return a false value, indicating that the process of computing the next batch of *Num_PB* convolutions is completed and the process of computing the next batch of *Num_PB* convolutions can commence.

Each PB actually implements *Calc_Next_NZPT()* function. This function determines on-the-fly, on a clock by clock basis, the next non-zero valued product term that should be computed, based on supplied compressed IFM bundle, *CIFM[x:Sh:x:Sh+Kernel_Width-1][y:Sv:y:Sv+Kernel_Height-1][:]:*, and compressed convolutional kernel, *CKM[kn][:]:[:]:*. If there are no more non-zero product terms *Calc_Next_NZPT()* function returns zero value, which is an indication that the PB has finished computing the convolution operation associated with it.

Please notice that the number of non-zero valued product terms present in individual convolution operation depends on the number of non-zero values in the input feature map bundle and associated convolutional kernel. Since all PBs operate on the same input feature map bundle, the number of non-zero IFM points is identical for all PBs. However, the number of non-zero convolutional coefficients can vary from one convolutional kernel to another. If this would be allowed, computing efficiency of Algorithm 3 could be severely degraded, because in this case, the number of non-zero product terms each PB must compute could vary significantly. In this scenario, all PBs that have already finished with computing their associated convolution would have to wait for the slowest performing PB to complete its calculation process. In order to limit this variation, we can enforce that each convolutional kernel from a given convolutional layer has to have an identical number of non-zero valued coefficients after pruning is performed. In other words, during pruning identical number of convolutional coefficients will be removed in every convolutional kernel from a given convolutional layer. Experiments that authors have conducted seem to indicate that this constraint is not significantly limiting the previously reported maximum achievable pruning rate for all CNN networks used in [3,10,52,53].

Please notice that even having convolutional kernels with an identical number of non-zero coefficients will not result in identical convolution computation times for different PBs. This is because the distribution of these non-zero coefficients will vary between different convolutional kernels, and when these kernels are correlated with the IFM bundle they could still have an unequal number of non-zero valued product terms to compute. In Section 4.4 a detailed analysis of this effect on the overall CNN processing efficiency of the CoNNA architecture will be presented.

Let us compare the number of MAC operations required to compute one convolution using Algorithms 1–3. In the case of Algorithms 1 and 2, the number of operations is constant and equals

$$Num_MACs_Alg_{1/2} = Kernel_Width \cdot Kernel_Height \cdot Kernel_Depth \quad (1)$$

In case of Algorithm 3, the number of MAC operations is not constant, since it depends on the number of non-zero valued product terms that are present, which in turn depends on the number of non-zero values in input feature map bundle and associated convolutional kernel, as well as on their actual positions within uncompressed structures. However, on average, the number of required MAC operations to compute one convolution operation using “All zero-skipping” technique equals

$$Num_MACs_Alg_3 = Kernel_Width \cdot Kernel_Height \cdot Kernel_Depth \cdot P_IFMNZ \cdot P_KMNZ \quad (2)$$

where *P_IFMNZ* is the percentage of non-zero valued points in the input feature map bundle, and *P_KMNZ* is the percentage of non-zero valued convolutional coefficients in the convolutional kernel.

Theoretical speedup in computing one convolution using the “All zero-skipping” technique over the “No zero-skipping” approach equals

$$Convolution_Calculation_Speedup = \frac{Num_MACs_Alg_{1/2}}{Num_MACs_Alg_3} = \frac{1}{P_IFMNZ \cdot P_KMNZ} \quad (3)$$

Convolution calculation speedup clearly depends on the number of zeros that are present in the input feature map bundle and convolutional kernel. While the number of zeros in the input feature maps cannot be directly controlled and also depends on the input instance that is currently being processed by the CNN network, the number of zeros in the convolutional kernel depends on the pruning ratio that was achieved during CNN network pruning process. Using previously published research we can estimate the typical number of zeros in the input feature maps, which ranges from 25% to 88% [19], and the typical number of zeros in convolutional kernels, which ranges from 16% to 78% [43]. Based on these figures, the achievable speedup of computing individual convolutional layers using the “All zero-skipping” technique over the “No zero-skipping” approach ranges from 1.59 up to 37.88, which is clearly a significant improvement.

Computing output of a fully-connected layer is the second most computationally demanding CNN layer type, after the convolutional layer. All other standard layer types, including pooling, adding, concatenation, have a negligible computational load, compared to convolutional and fully-connected layers. Please notice that the computation of a fully-connected layer can be mapped to a convolutional layer computation in the following way. Each neuron from the fully-connected layer computes the weighted sum over the entire input feature map, using its own specific set of weights. However, this weighted sum computation can be seen as the special case of convolutional sum computation, where the convolution involves the entire input feature map and convolutional coefficients equal fully-connected neuron weight values. Therefore, computing fully-connected layer of *N* neurons is equivalent to computing *N* different convolutions spanning the entire input feature map. The result of processing a fully-connected layer will be an output feature map of dimension $1 \times 1 \times N$. This means that we could use Algorithms 1–3 to compute the output of a fully-connected layer also, configuring algorithm parameters in the following way: *OFM_Width* = 1; *OFM_Height* = 1; *Kernel_Num* = *N*; *Kernel_Width* = *IFM_Width*; *Kernel_Height* = *IFM_Height*; *Kernel_Depth* = *IFM_Depth*.

3.2. Overview of the CoNNA CNN accelerator architecture

The CoNNA architecture is designed to act as a standalone CNN hardware accelerator. It is designed as a standard soft-IP core, that can be easily integrated into contemporary System-on-Chip (SoC)

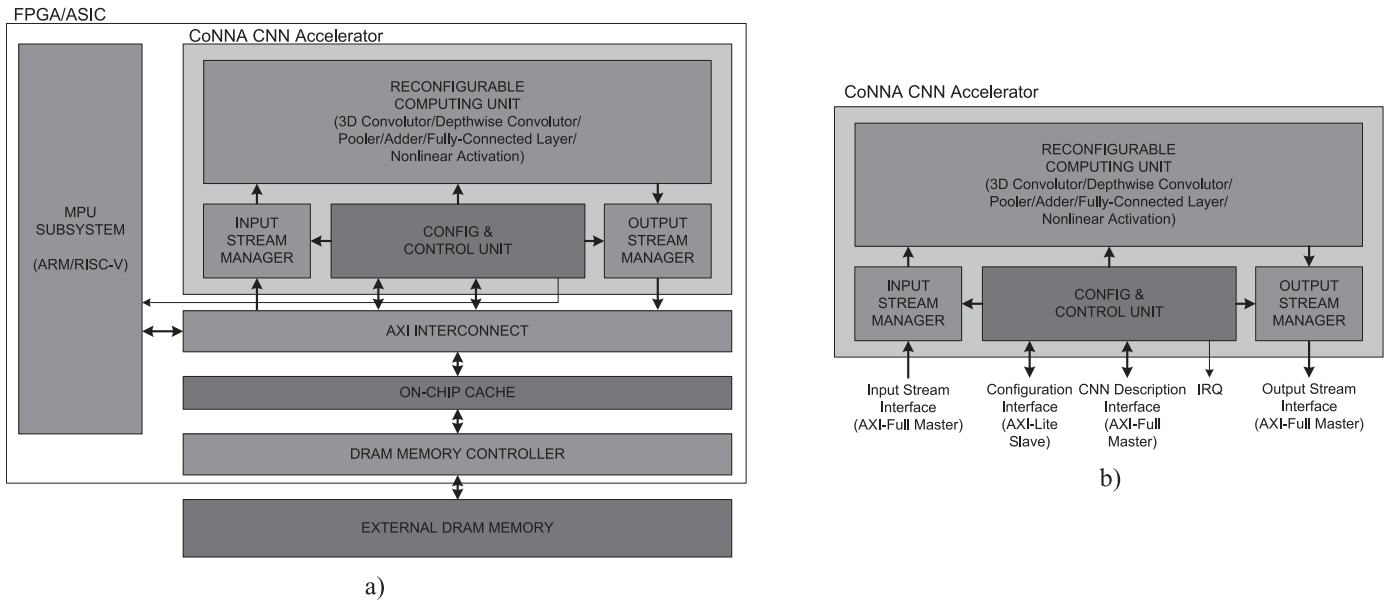


Fig. 3. CoNNA Compressed CNN Accelerator: a) Integration into SoC or PSoc; b) Top level architecture with available interfaces.

or Programmable System-on-Chip (PSoc) solutions. CoNNA uses standard AXI interfaces, so it can be easily connected with ARM or RISC-V microprocessors, using standard AXI-Interconnect modules, as shown in Fig. 3a.

Having a microprocessor within the system eases the control and configuration of the CoNNA CNN accelerator, but it must be stated that configuration and control of the CoNNA accelerator is a relatively simple process, so a pure hardware-based solution, with hardwired FSM controlling CoNNA, can also be used. However, in order for CoNNA to be fully functional, some kind of memory needs to be present in the system, which could be on-chip cache memory or off-chip operating memory, as shown in Fig. 3a. If the system can implement large enough on-chip cache memory, as shown in Fig. 3a, of an order of several tens of megabytes, which is today possible even if we use FPGA devices, the result would be a highly power-efficient solution. In this configuration, all CNN topology data, as well as all intermediate feature maps would be stored inside the on-chip cache. This would significantly reduce required data movement between external memory and the CoNNA accelerator and consequently lower power consumption since most of the energy in data processing systems is spent in moving data between external DRAM memory and processing system [47]. However, for low-cost solutions, this configuration will probably not be possible, so in this case, instead of using an on-chip cache, the system would have to include external operating memory, usually implemented as DRAM memory, as shown in the Fig. 3a.

The top-level architecture of the CoNNA compressed CNN accelerator is shown in the Fig. 3b. The CoNNA CNN accelerator is composed of the following four modules:

- Reconfigurable Computing Unit (RCU) – used to perform all computations defined by different layers from the CNN network (including convolutional, depthwise convolutional, pooling, concatenation, adding and fully connected layers), exploiting the sparsity of kernel and feature maps. RCU module is further composed out of a number of Processing Block modules.
- Input Stream Manager (ISM) – used to supply all input data (configuration, kernel map, input feature map), in compressed format, coming from external DRAM memory or on-chip cache memory to the appropriate internal modules of the RCU module.

- Output Stream Manager (OSM) – used to format, compress and stream output feature map data, generated by the RCU as it processes CNN layers, to the external DRAM memory or on-chip cache memory.
- Configuration and Control Unit (CCU) – used to control the operation of the CoNNA CNN accelerator and enable interfacing with surrounding logic.

The CoNNA CNN accelerator uses four AXI interfaces to communicate with surrounding on-chip modules:

- Input Stream Interface (ISI) – AXI-Full interface used to stream input data, including input image, intermediate input feature maps and kernel maps for selected CNN network, to the CoNNA accelerator module for processing. CoNNA acts as the master of the ISI interface. The other side of the ISI interface is connected either to the on-chip cache module or to the DRAM memory controller module.
- Output Stream Interface (OSI) – AXI-Full interface used to stream output feature maps data to the on-chip cache or external DRAM memory. CoNNA acts as the master of the OSI interface. The other side of the OSI interface is connected either to the on-chip cache module or to the DRAM memory controller module.
- CNN Description Interface (CDI) – AXI-Full interface used to load structural information about the CNN network that is being accelerated. CDI interface is implemented using standard AXI-Full interface protocol. CoNNA acts as the master of the CDI interface. The other side of the CDI interface is connected either to the on-chip cache module or to the DRAM memory controller module.
- Configuration Interface (CI) – AXI-Lite interface used to configure and control the operation of the CoNNA CNN accelerator. CI interface is implemented using standard AXI-Lite interface protocol. CoNNA acts as the slave of the CI interface. The other side of the CI interface is connected either directly to the host processor or hardwired FSM or to some AXI interconnect module.

Furthermore, the CoNNA accelerator has an interrupt request output port, which is used to signal to the surrounding modules that CoNNA has finished processing one input instance, and that the instance classification data is available. Since the CoNNA

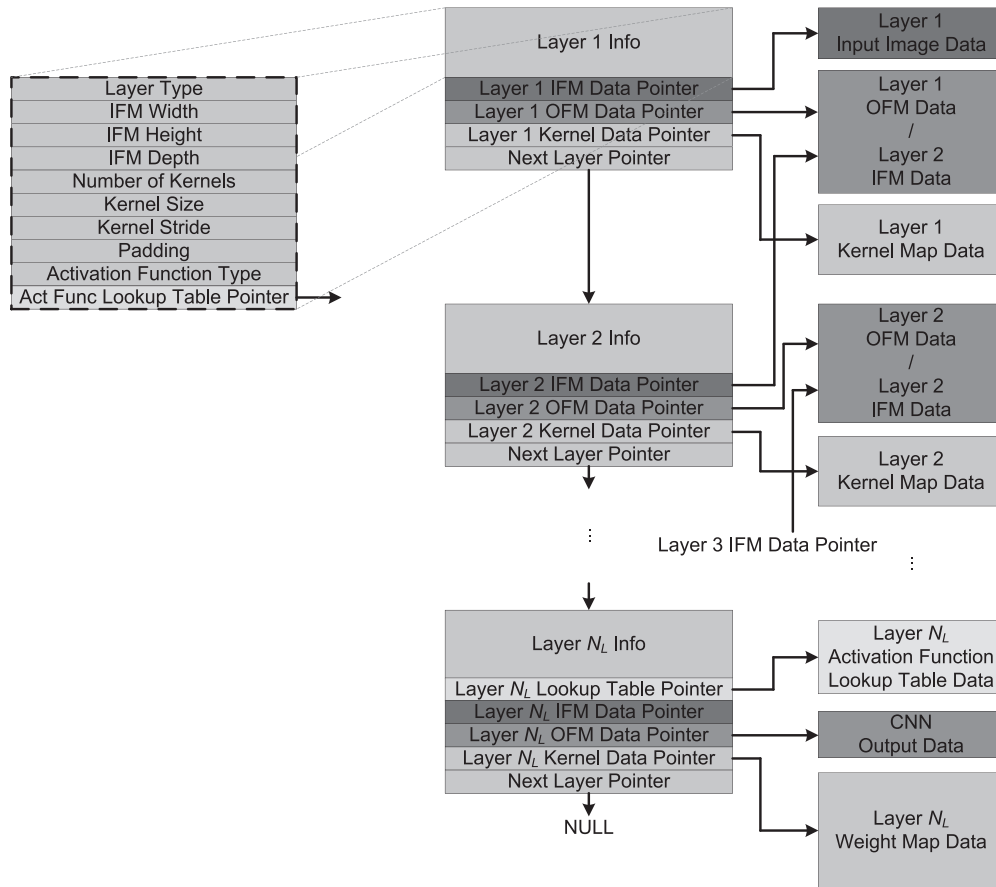


Fig. 4. Linked list representation of target CNN structural description that will be accelerated by the CoNNA CNN accelerator.

CNN accelerator uses AXI interfaces, integration within ARM/RISC-V based SoCs or PSoCs is greatly simplified. All data that is being processed or generated by the CoNNA accelerator is stored either in the internal on-chip cache memory or external DRAM memory, depending on the available memory resources in the system.

The operation of the CoNNA CNN accelerator is configured and controlled using a set of internal registers, as well as several data structures required for correct operation, which are stored either in the on-chip cache memory or external DRAM memory. The CoNNA CNN accelerator operates on the following data structures:

- CNN Structural Description (CNNSD) list – this is a linked list of nodes, each one describing one layer from a particular CNN network that the CoNNA accelerator should process.
- Input Instance Buffer (IIBs) – memory buffer, used for storing input instances that should be processed by the CoNNA CNN accelerator.
- Feature Map Buffers (FMBs) – a number of memory buffers, used for storing input feature maps and output feature maps data that are processed by various layers from the target CNN network. The minimum number of FMB buffers equals two, one for storing input feature map that is being processed by the current CNN layer and another for storing output feature map that is being generated by the current CNN layer. This configuration is sufficient in case the target CNN network topology doesn't contain adding or concatenation layers, like in the case of AlexNet, VGG or MobileNet V1 CNN architectures. However, if the target CNN network topology contains adding or concatenation layers, which is, for example, the case with ResNet, Inception and NASNet architectures, then additional FMB buffers are necessary.

- Classification Data Buffer (CDB) – memory buffer used for storing classification data for the last input instance that was processed by the CoNNA CNN accelerator. The output feature map generated by the final layer of the target CNN network is stored in this buffer. This data represents the classification information about the instance that was processed by the CNN network, and it can be used by higher application levels within the complete system.

CNNSD linked list, shown in the Fig. 4, defines the topology, together with various parameter values, of the target CNN network that should be accelerated by the CoNNA CNN accelerator. CNNSD list is composed of a number of nodes, the exact number being equal to the number of layers in the target CNN network, where each node describes the characteristics of one particular layer from the selected CNN network.

Every CNNSD node, shown in Fig. 4, contains the following fields:

- Layer Info – this field, which has multiple subfields, is used to specify details about the current CNN layer that should be processed by CoNNA. Layer Info field is composed of the following subfields:
 - Layer Type – this field specifies the type of the current layer, valid values being: convolutional, depthwise convolutional, pooling (average or max pooling), adding, concatenation, and fully-connected.
 - IFM Width – this field specifies the width of the input feature map, expressed as the number of IFM points.
 - IFM Height – this field specifies the height of the input feature map, expressed as the number of IFM points.

- IFM Depth – this field specifies the depth of the input feature map, expressed as the number of IFM points.
- Number of Kernels – this field specifies the number of kernels used to process IFM data by the current convolutional layer. This number also determines the depth of the OFM generated by the current convolutional layer. This field is valid only if the “Layer Type” field is set to “convolutional” or “depthwise convolutional”.
- Kernel Size – this field specifies the horizontal or vertical size of the square convolutional kernel, or pooling area, expressed as the number of KM points. This field is valid only if the “Layer Type” field is set to “convolutional”, “depthwise convolutional”, “max pooling” and “average pooling”.
- Kernel Stride – this field specifies the stride of the convolutional kernel or pooling area. This field is valid only if the “Layer Type” field is set to “convolutional”, “depthwise convolutional”, “max pooling” and “average pooling”.
- Padding – this field specifies if padding should be used when processing IFM data with the current CNN layer.
- Activation Function Type – this field specifies the type of activation function to be used, valid values being: “none”, “ReLU” or “arbitrary”. In the case of the “arbitrary” type, additional Activation Function Lookup Table Pointer field points to the beginning of the activation function lookup table content memory block. This block is loaded into the CoNNA’s Arbitrary Non-Linear Activation Function Calculator module, located within the RCU module, to specify the exact shape of the non-linear activation function that should be used within the current CNN layer. In the case of “none” and “ReLU” values, Activation Function Lookup Table Pointer is ignored.
- Input Feature Map Data Pointer – this field specifies the base address of the memory block, which holds the values of the input feature map of the current CNN layer.
- Output Feature Map Data Pointer – this field specifies the base address of the memory block, which will store the values of the output feature map, generated by the current CNN layer.
- Kernel Data Pointer – this field specifies the base address of the memory block, holding the kernel coefficients, in case of a convolutional or depthwise convolutional layer, or weight values in case of a fully connected layer.
- Next Layer Pointer – this field specifies the address of the next CNNSD node, which describes the characteristics of the subsequent layer from the CNN network that is being accelerated. If the value of this field is **NULL**, this is an indication that the current CNN layer is actually the final layer of the CNN and that the CoNNA accelerator should signal completion of the current input instance classification, after it has processed this final layer.

Besides CNNSD linked list, IIB, FMB and CDB buffers, the CoNNA CNN accelerator has the following set of internal registers, that are used to configure and control its operation. All registers are 32-bit, and are accessed using the CI AXI-Lite interface of the CoNNA CNN accelerator:

- Control Register – used to control the operation of the CoNNA CNN accelerator.
- Status Register – used to monitor the operation of the CoNNA CNN accelerator.
- CNNSD Pointer – register holding the base address of the CNNSD linked list, where the first CNNSD node is located. The CoNNA CNN accelerator uses this address value to fetch the information about the structure of the first and all subsequent layers of the CNN network that is being accelerated.

Because CNN’s structural information is stored in a form of a linked list, on-the-fly selection of target CNN network to be accel-

erated is possible when using the CoNNA CNN accelerator, without the need to modify and re-implement the accelerator itself. This situation is shown in the Fig. 5. There can be a number of CNN topology description lists, sitting in the operating memory to which the CoNNA accelerator is connected. By simply writing the correct base address of the target CNN description list in the CNNSD Pointer register, the user can easily select the desired CNN network that should be processed by the CoNNA accelerator. This selection can be done on-the-fly, without any need for hardware reprogramming or reconfiguration. Switching between different CNN networks that should be accelerated is extremely fast, requiring only one write to CoNNA’s internal register (CNNSD Pointer). This feature of the CoNNA accelerator is of great importance in the case of applications where rapid, dynamic switching of CNN networks is necessary. It is worth noting that many previously proposed CNN accelerators, particularly accelerators designed using HLS techniques, cannot support this kind of functionality.

Furthermore, on-the-fly modification of accelerated CNN network topology is also possible when using CoNNA CNN accelerator, removing or adding CNN layers or changing parameters of existing layers, by a simple modification of the target CNN description linked list. For example, in the Fig. 5 CNN description linked list for M^{th} CNN network could be modified by replacing the existing description of Layer 2 with the description of some other layer, for example, Layer 2*. This can easily be done, by simply rewriting existing values of the “Next Layer Pointer” field from Layer 1 node and IFM Data Pointer field from Layer 3 node. After these modifications, the topology of M^{th} CNN network will be modified to include a new layer, Layer 2*, instead of existing Layer 2. As can be seen, this modification is very quick, requiring only a couple of memory accesses to the operating memory where the CNN description list is stored. Therefore, rapid, dynamic modification of target CNN topology is also supported by the CoNNA CNN accelerator, which can be of particular interest when working with hierarchical CNNs [48–50].

CoNNA processes the CNN network in a sequential manner, one layer at a time. It uses information about the current CNN layer to be accelerated, stored in the appropriate node of the associated CNN description linked list, to reconfigure the RCU unit and perform all necessary computations defined within that CNN layer. When all computations defined in the current layer are completed, output feature map data generated by the current layer is stored in the appropriate memory buffer and CoNNA can proceed with the next CNN layer from the supplied CNNDS linked list. This operation is repeated until a final node from the CNNDS linked list is processed, indicating the end of processing of the current input instance by selected CNN. The process of sequential processing of CNN layers employed by the CoNNA CNN accelerator is shown in the Fig. 6.

Please note that although CoNNA processes CNN layers sequentially, layer by layer, computations defined within each CNN layer are executed concurrently, using available Processing Block modules from the RCU module. The way how this is done was presented at the beginning of Section 3, and will also be explained in more detail in Section 3.2.1.

3.2.1. Details of RCU module

RCU module is the central module of the CoNNA CNN accelerator. It is used to perform all necessary numerical calculations, defined by different CNN layers. RCU module, shown in Fig. 7, is designed as a coarse-grained reconfigurable hardware module, allowing easy, fast, dynamic, on-the-fly reconfiguration, in order to create different dataflows, optimized for processing particular CNN layer type.

RCU module is composed of a number of Processing Block modules and one Activation Function Calculator (AFC) module. PBs are

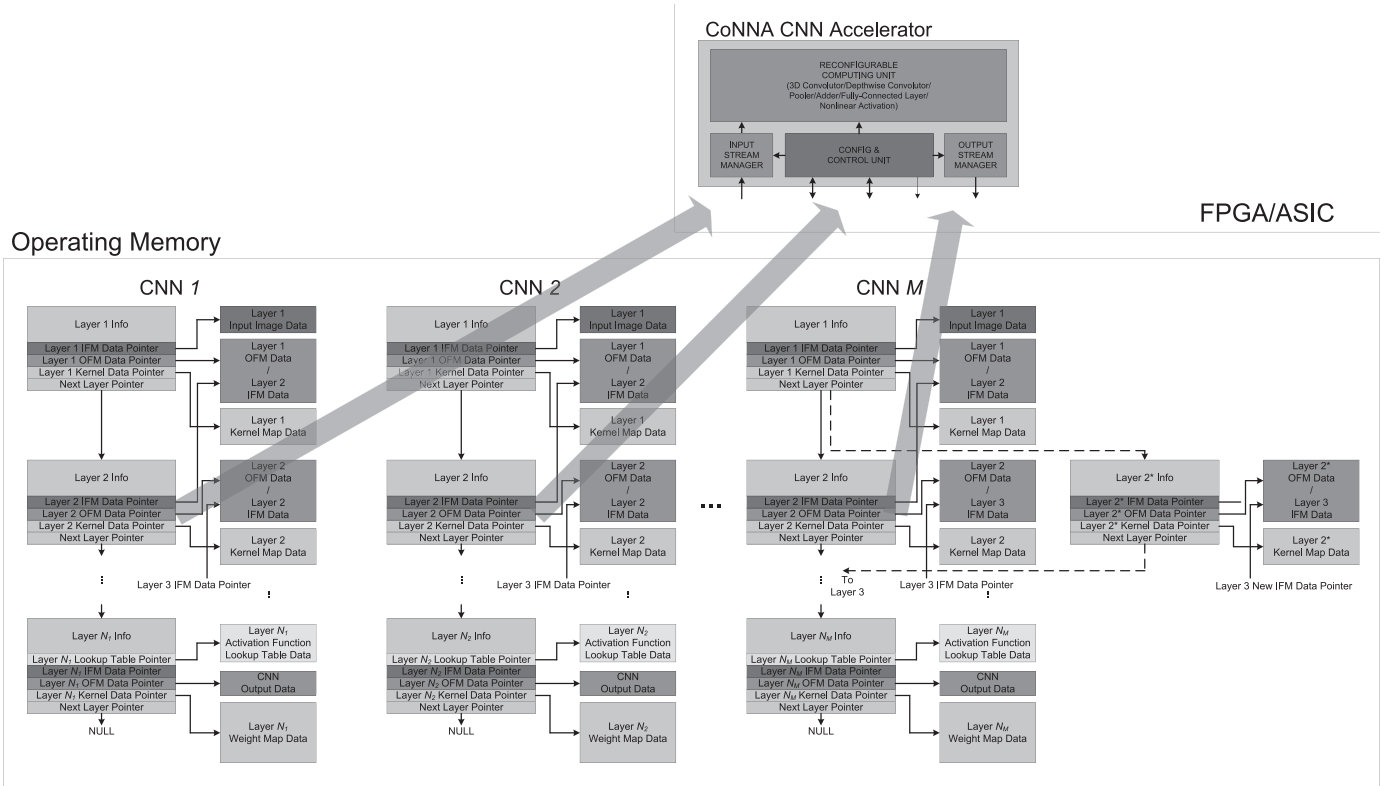


Fig. 5. Principle of on-line switching between different CNNs and accelerating CNNs with dynamic topology.

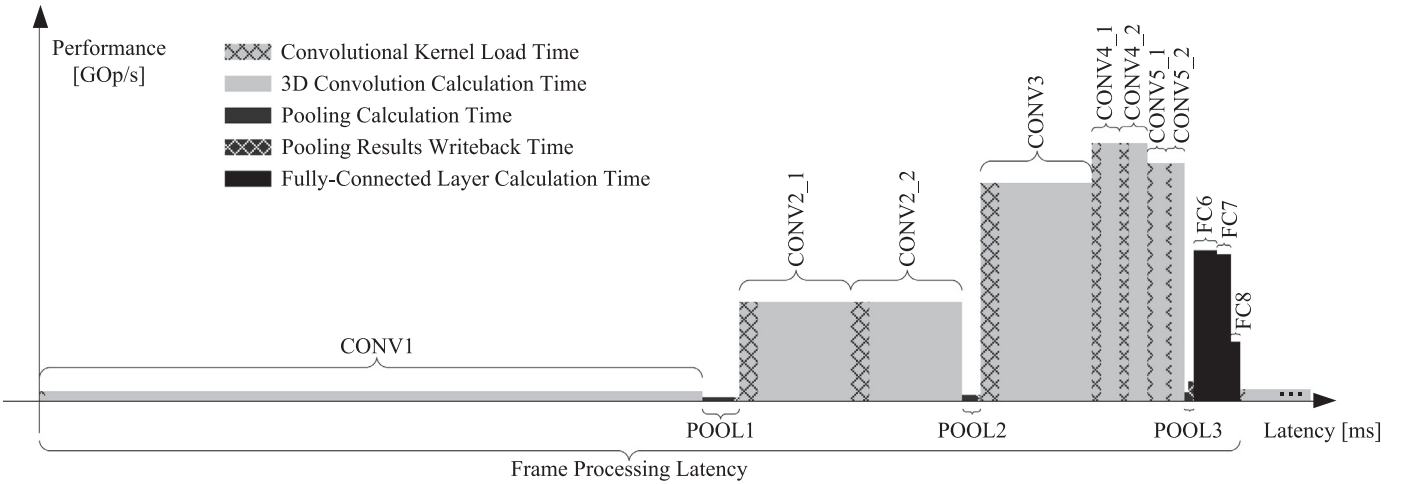


Fig. 6. Principle of sequential layer processing during CNN acceleration implemented by CoNNA.

able to perform all required CNN layer processing operations, using compressed input feature map and kernel map data, implementing the “All zero-skipping” technique, to increase CNN computation performance. AFC module is used for post-processing data coming from PB modules. AFC module contains a number of Rectified Linear Units (ReLU), implementing the Rectified Linear activation function and one Arbitrary Non-Linear Activation Function Calculator unit, capable of implementing arbitrary non-linear activation functions, which are mainly used in fully-connected CNN layers.

As already mentioned in Section 3.1, during convolutional layer processing CoNNA allocates computation of one complete convolution to a single PB. Multiple convolutions, operating on the same IFM bundle are being calculated in parallel, by different PBs, as shown in the Fig. 7b, and previously described by Algorithm 3.

In the Fig. 7b distribution of different convolutions over available PBs, in case of processing convolutional layer with 6 different convolutional kernels, is presented. Please notice that if the number of convolutional kernels is smaller than the number of PBs, some of the PBs will be inactive. Also note that the amount of parallelism is limited by the number of available PBs, since their number determines the partial unrolling factor that will be used to unroll loop L3 from Algorithm 3, Num_PB . However, for most standard CNN networks [3,10,52,53] number of different convolutional kernels specified in convolutional layers rises sharply as we move deeper inside the CNN network, and quickly reach values above 100, so this constraint is usually not so severe.

The detailed architecture of Processing Block is shown in the Fig. 8. PB contains non-zero product term detection logic, imple-

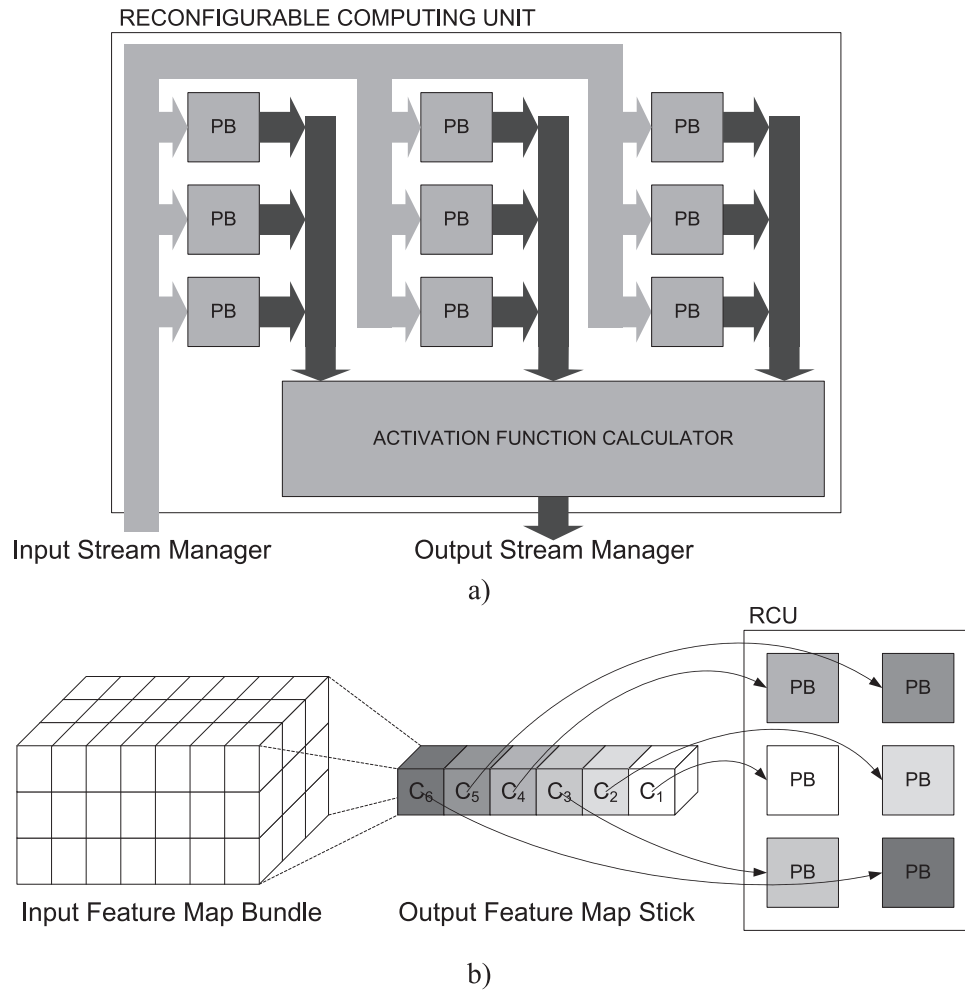


Fig. 7. Details of the RCU Module: a) top-level architecture of the RCU Module; b) mapping individual convolutions to available PB units.

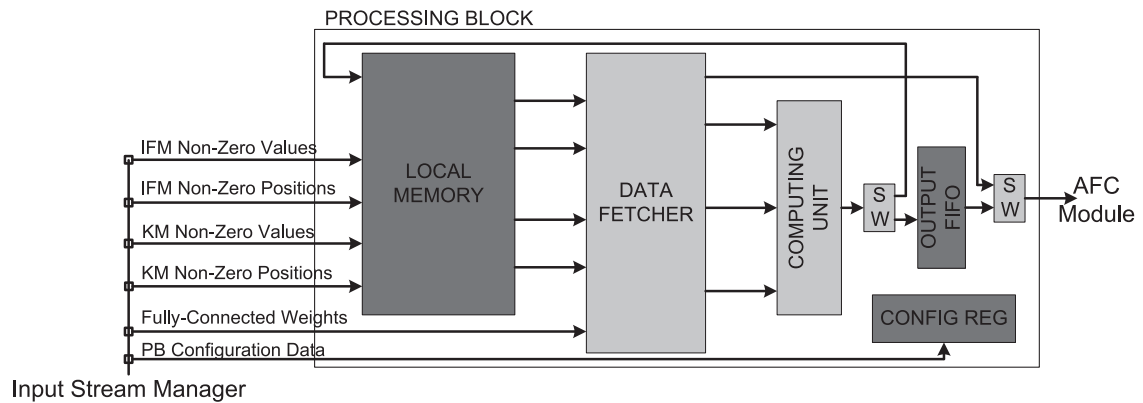


Fig. 8. The architecture of the processing block module.

mented inside the Data Fetcher (DF) module, which allows the execution of numerical computations only when their result will be different from zero, skipping all unnecessary (zero-outcome) computations, resulting in a significant speedup of the CNN calculation process. Please notice that the DF module actually implements the *Calc_Next_NZPT()* function from Algorithm 3, directly in hardware. PB also contains four local memories, located in the Local Memory module (LM), used for storing data from selected convolutional kernel and IFM bundle, in compressed format. Two memories are used to store all non-zero valued convolution kernel co-

efficients and their actual positions in the original uncompressed kernel. Remaining two memories do the same for the IFM bundle that is currently being processed by the PB module. Please notice that all data stored in these four memories are stored in a serialized format, as the result of flattening loops L4-L6 from Algorithm 2, using the "Stick First" approach described during the analysis of Algorithm 2. Computing Unit (CU) performs all necessary operations on the selected kernel and IFM value pairs, supplied by the DF module. It contains one MAC unit, together with some additional logic, for enabling different compute dataflows,

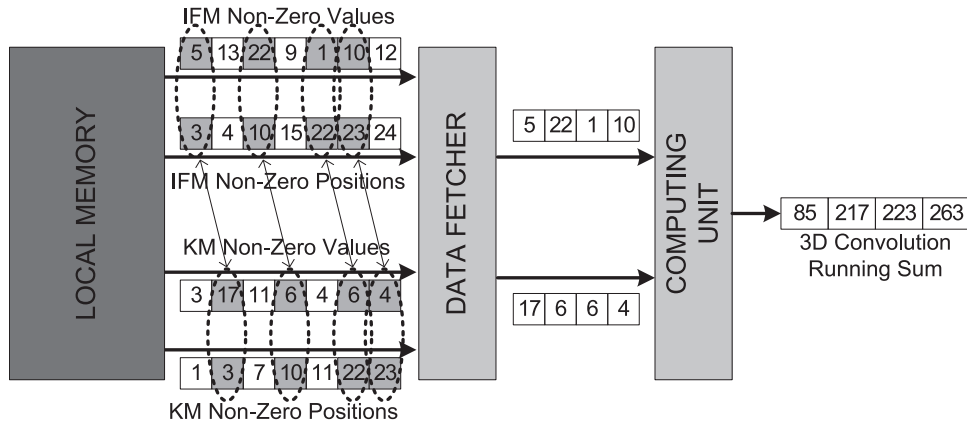


Fig. 9. Zero-valued product term skipping principle of operation.

depending on the type of CNN layer that is being processed by the CoNNA CNN accelerator. Output FIFO is used to synchronize processing steps performed by different PBs since different PBs can require different periods of time to calculate their associated convolutions. This can occur because of the possible difference in the actual number of non-zero valued product terms that are present in different convolutions, as it was described during the presentation of Algorithm 3. Finally, the Configuration Register is used to specify the desired PB operating mode that should be used. PB module supports several datapath configurations, which enable efficient processing of convolutional, pooling, fully-connected and adding CNN layer types.

The principle of the “All zero-skipping” technique used to skip all zero-valued product terms, used by the PB module during the convolution calculation process, is shown in the Fig. 9. LM module streams information about the positions of kernel and IFM bundle non-zero values to the DF module. DF module detects on-the-fly the next kernel/IFM non-zero valued pair, implementing the “All zero-skipping” technique, using the information about the kernel/IFM non-zero value positions in the uncompressed data structures, and passes it to the CU module for processing. This is accomplished by a parallel search for the next coincident non-zero valued position in both KM and IFM data streams, within a specified search window, which is 16 elements wide in the case of the CoNNA architecture. This operation is repeated until all data is used, and all relevant, non-zero valued product terms are accumulated within the CU module, to obtain the final value of convolution calculation operation.

3.2.2. Details of ISM and OSM modules

ISM module is used to stream input data to the RCU module. Input stream data consists of PB configuration data, input image data or input feature map data, and convolutional kernel coefficient values or fully-connected weight values, depending on the type of CNN layer that is currently being processed. All CNN-related data, which include all above-mentioned data except PB configuration data, is received either from on-chip or external DRAM memory in compressed format, which is then either stored inside the Input Stick Buffer (ISB) module, in case of the input feature map data, or routed directly to the appropriate LM module inside the selected PB module, in case of convolutional coefficients or fully-connected weights. Please notice that the CoNNA CNN accelerator is designed to process all incoming CNN-related data in compressed format, so there is no need to decompress it. Fig. 10 presents details about the internal organization of the Input Stream Manager module, with all major sub-modules shown.

Input Stream Manager is composed of the following major modules.

- Input Stick Buffer (ISB) memory – used for storing cached IFM sticks in compressed format.
- Stick Valid (SV) memory – used to store information is the IFM stick located at the same position in ISB memory valid for reading or not.
- Read Controller (RC) – used to read selected compressed IFM stick from ISB memory and transfer it to the RCU module, more specifically into two local memories located inside the LM module of each PB module.
- Write Controller (WC) – used to write compressed IFM stick from external DRAM memory or on-chip cache memory to appropriate position inside ISB memory.
- Input Stream Router (ISR) – used to select which input data stream will be sent to the RCU module. Two possible input data stream sources are available:
 - First, coming from the RC module, used when compressed IFM sticks are being transferred to the RCU.
 - Second, coming directly from Input Stream Interface, used when convolutional kernel map or fully-connected weight map data is being transferred to the RCU module.

The central module of ISM is the Input Stick Buffer memory module. This memory is used for storing selected compressed IFM sticks from the current input feature map, which is being processed by the current CNN layer. During input feature map processing by the convolutional layers, IFM sticks are being repeatedly reused during the calculation of adjacent convolutions, as convolutional kernels slide over input feature map. This will always be the case if the horizontal and vertical stride values, S_h and S_v , are smaller than the corresponding horizontal and vertical kernel sizes, $Kernel_Width$ and $Kernel_Height$. This opens a possibility to minimize data movement between the CNN accelerator and external memory by caching IFM sticks that will be reused during the adjacent convolutions computation process. ISB memory module acts as this local cache of selected IFM sticks. ISB stores selected compressed IFM sticks, $1 \times 1 \times S_D$ sections of the IFM, where S_D is the number of non-zero valued IFM points that are present in the current IFM stick, which will be used in the upcoming convolution calculation operations, as shown in the Fig. 10b. Please notice that the 3D cube from Fig. 10b, which represents the content of the ISB memory module, is actually toothed since different IFM sticks can contain a different number of non-zero valued points.

Each time the same IFM stick is needed in the convolution calculation operation, instead of re-fetching it from external DRAM memory, it is fetched from the ISB module, reducing the number of data transfers from external DRAM memory, thus saving power. Once all convolution operations involving given IFM stick are completed, that IFM stick can be removed from the ISB module and

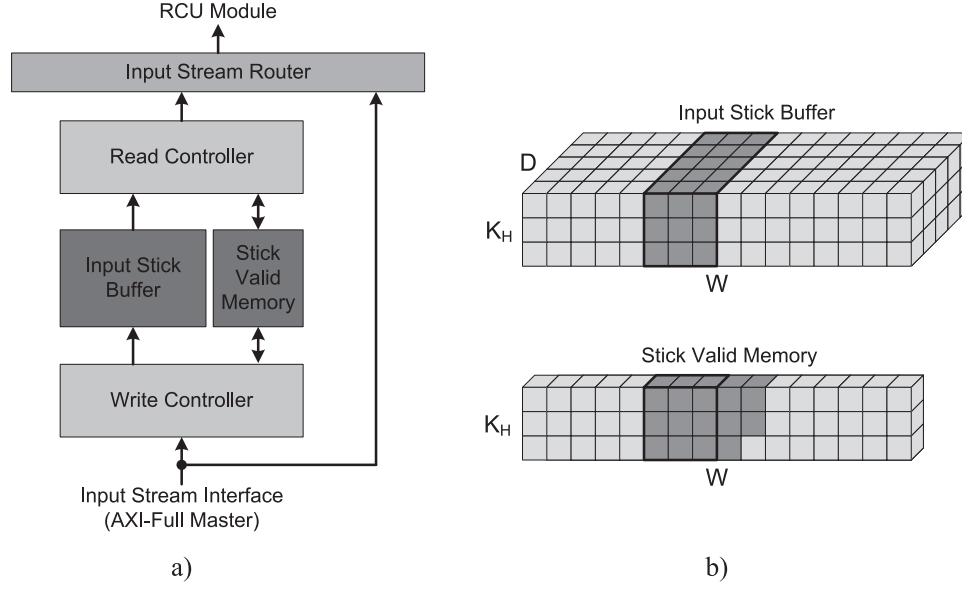


Fig. 10. Input stream manager: a) Top level ism modules; b) Organization of ISB and SVM memories.

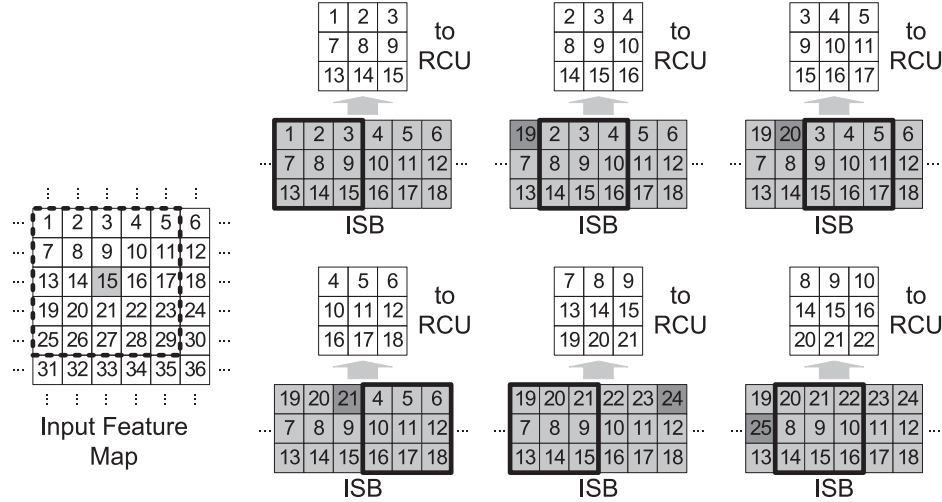


Fig. 11. Principle of ISB memory operation.

the next IFM stick can be loaded in its place from external DRAM memory.

Read Controller and Write Controller modules are charged with correct manipulation of IFM sticks stored inside the ISB cache memory module. WC module sweeps the input feature map line by line, as shown in Fig. 11, and writes IFM sticks in appropriate positions inside the ISB memory module. Please notice that the required size of ISB memory is significantly smaller than the size of the complete input feature map that is being processed by the current convolutional layer. ISB memory size, measured in the number of IFM points, that is sufficient to correctly cache IFM memory sticks from a given CNN network equals

$$\begin{aligned}
 \text{ISB_Memory_Size} = & \max_{i \in \text{layers}} \{ \text{IFM_Width}_i \} \\
 & \cdot \max_{\substack{i \in \text{layers} \\ j \in \text{kernels}}} \{ \text{Kernel_Height}_{i,j} \} \\
 & \cdot \max_{\substack{i \in \text{layers} \\ 0 \leq j < \text{IFM_Width}_i \\ 0 \leq k < \text{IFM_Height}_i}} \{ S_{D_{i,j,k}} \}
 \end{aligned} \quad (4)$$

Required ISB memory size clearly depends on the size of the input feature map that is being cached and the percentage of non-zero valued IFM points, but is significantly smaller than the memory required to store the complete IFM map, because *Kernel_Height* parameter is always significantly smaller than the *IFM_Height* parameter.

On the other hand, the RC module reads selected IFM sticks from ISB memory and transfers them to the RCU module. RC module reads IFM sticks in a different order from the order WC module uses to write them in the ISB memory. While WC traverses input feature map horizontally, loading complete IFM line before moving to the next one, RC traverses ISB memory in the more localized manner, defined by the frontal shape of the convolutional kernel, as shown in the Fig. 11 in the case of 3×3 kernel.

Please notice that the speed at which the WC module writes IFM sticks into ISB memory can differ from the speed at which the RC module reads IFM sticks from ISB memory. This can happen because data processing throughputs of the memory subsystem and RCU module can be different. Therefore, a mechanism that ensures the consistency of ISB memory content needs to be devised. This is the purpose of the Stick Valid memory module. SV memory is

IFM_Width-Kernel_Height-1 bits large. For each IFM stick from ISB memory there is a corresponding bit in the SV memory, indicating is that particular IFM stick valid or not. Each time WC module wants to write a new IFM stick into selected location inside the ISB memory it first checks is the corresponding bit from the SV memory cleared, which indicates that all convolutions which use IFM stick currently stored at the selected location inside ISB memory have been computed, and it can be replaced with a new IFM stick. If this is the case, WC writes a new IFM stick in the selected location inside ISB memory and sets the corresponding bit inside SV memory. Otherwise, the WC module needs to wait until the corresponding SV memory bit is cleared. When the RC module wants to read IFM stick from ISB memory it first checks is the IFM stick at selected ISB memory location valid or not, by checking the corresponding bit from the SV memory. If the bit is set, this means that the IFM stick is valid and RC can transfer it to the RCU module, otherwise the RC module has to wait until the WC module certifies that the IFM stick data is valid.

The operation of ISB memory is shown in the Fig. 11, which shows a snapshot of the input feature map, 6×6 IFM sticks in size. Please notice that there is a third dimension to the IFM since its depth is always bigger than one, but for clarity, it was omitted. If we assume that this IFM is being processed by a 3×3 kernel with horizontal and vertical strides of 1, then each IFM stick will be used in the process of calculating of up to nine different convolutions, as shown in the Fig. 11 for stick number 15. Without the ISB module, each IFM stick would have to be loaded from DRAM memory up to nine times, but by using ISB memory it is only necessary to load each IFM stick exactly once from external DRAM. This means that for the example shown in the Fig. 11, it is possible to reduce DRAM memory traffic when transferring IFM data, up to nine times. Fig. 11 also shows the concept of replacing already used IFM sticks within the ISB module with the new ones. The new IFM stick that is written in the ISB memory by the WC module is shown in dark gray color in the Fig. 11.

OSM module is used to collect output data from the RCU module, pack it into appropriate blocks in the form of output feature map sticks, in order to maximize the data throughput between the accelerator and the external DRAM memory, and compress these OFM sticks on-the-fly. ZRL encoder module is used to compress sequences of zeros within the OFM sticks, which will contain a significant number of zero values if the ReLU activation function is being used. Compressed OFM sticks, containing only non-zero valued OFM points and their locations within original, uncompressed OFM data, are then written into either external DRAM memory or internal on-chip cache memory, as shown in the Fig. 3. For more details about the encoding algorithm that is used to compress OFM data, please refer to Horowitz [46].

4. Experiments

The performance of the CoNNA architecture was compared with the following seven previously proposed CNN hardware accelerators. MIT's Eyeriss [19] is an ASIC CNN hardware accelerator and it was chosen because it is de-facto a standard reference for comparison, used in many CNN hardware accelerator papers. NullHop [32] is an FPGA-based CNN accelerator using the "IFM zero skipping" technique. NEURAghe [23] is a closely coupled CNN hardware accelerator with ARM processor employing a cooperative heterogeneous computing approach to CNN acceleration, intended to be used with ARM-based FPGA families, like Xilinx Zynq or ZynqUltrascale families. CNN_A1 [29] is a reconfigurable CNN accelerator based on using CNN-specific Instruction Set Architecture, and embedding parallel computation and data reuse parameters in the instructions. fpgaConvNet CNN accelerator [25] is the accelerator created using HLS techniques targeting FPGA devices. Aristotle ac-

Table 1
Characteristics of the CNNs used in comparison experiments.

CNN Architecture	No. of Layers (Conv, Pool, Add, Fully-Conn)	No. of Parameters	
[Millions]	No. of Operations		
[GOps]			
AlexNet [3]	(16, 5, 0, 3)	60.93	1.45
VGG-16 [10]	(16, 5, 0, 3)	138.35	30.95
VGG-19 [10]	(19, 5, 0, 3)	143.66	39.28
GoogLeNet [52]	(21, 5, 0, 1)	6.97	3.16
ResNet-18 [53]	(17, 2, 8, 1)	11.51	3.59
ResNet-50 [53]	(49, 2, 16, 1)	25.50	7.72

celerator, originally developed by a startup company Deephi [30], now is the part of the IP portfolio of the largest FPGA manufacturer Xilinx, who recently bought DeePhi. Finally, NVDLA [51] is one of the first open-source based CNN accelerators coming from the industrial sector, developed by NVIDIA, the current leader in the deep learning acceleration field. The performance of the CoNNA CNN accelerator was compared with seven previously proposed CNN accelerators using six well-known CNN networks, AlexNet [3], VGG-16, VGG-19 [10], GoogLeNet [52], ResNet-18 and ResNet-50 [53]. Relevant data for each CNN network used in experiments are presented in Table 1.

Configurations of all reference CNN accelerators that were used in performance comparison experiments are presented in Table 2. Configurations of Eyeriss, NullHop, NEURAghe, CNN_A1, fpgaConvNet, and Aristotle CNN accelerators are the ones that were used in original papers [19,23,25,29,30,32], in order to enable easy comparison. As for the NVDLA [51], it comes with an Excel sheet estimator, which allows specifying different configuration options. We have used the one presented in Table 2 since the CoNNA accelerator is mainly intended for usage in low-cost, edge-computing devices with limited computing resources.

4.1. Generating CoNNA CNN architecture instances needed for comparison with previous work

The CoNNA CNN accelerator was modeled as a standard soft-IP core, using SystemVerilog hardware description language, with many configuration parameters, allowing easy generation of different CoNNA instances with the same underlying architecture. In order to create a complete CNN acceleration system based on the CoNNA accelerator following four steps, shown in Fig. 12, need to be performed:

1. Training and pruning selected CNN network – for this task, any fine-grained pruning algorithm can be used, and also any available deep learning framework, like Caffe, TensorFlow, Keras, and Matlab, can be used to train and prune selected CNN network. We have implemented the fine-grained pruning algorithm described in Section 2 within Keras deep learning framework, and used Keras for training and pruning CNN networks presented in Table 1.
2. Selecting optimal fixed-point number representation and generating a description of pruned CNN in a format recognizable by the CoNNA CNN accelerator – pruned and retrained floating-point CNN model needs to be converted into a 16-bit fixed-point model because CoNNA uses fixed-point arithmetic blocks for implementing all arithmetic operations required to process CNN network. After this conversion, the model which is still represented in a format that depends on selected deep learning framework needs to be converted into a CNN structural description linked list, presented in Section 3.1, in order to be "understandable" by the CoNNA CNN accelerator. This CNN structural description linked list will actually be generated in a binary format, which can be downloaded into operating memory

Table 2
Important Characteristics of reference CNN accelerators used for comparison.

Accelerator/Parameter	Supported CNN Layers	Number of MAC Units	Operating Frequency	Arithmetic Precision
Eyeriss [19]	Conv only	168	200 MHz	16-bit
NullHop [32]	Conv, Pooling, Fully-Connected	128	60 MHz	16-bit
NVDLA [51]	All	32	100 MHz	16-bit
NEURAghe [23]	All	864	140 MHz	16-bit
CNN_A1 [29]	Conv, Pooling, Fully-Connected	216	150 MHz	16-bit
fpgaConvNet [25]	Conv, Pooling, Fully-Connected	164, 198	125 MHz	16-bit
Aristotle [30]	Conv, Pooling, Fully-Connected	198	214 MHz	8-bit

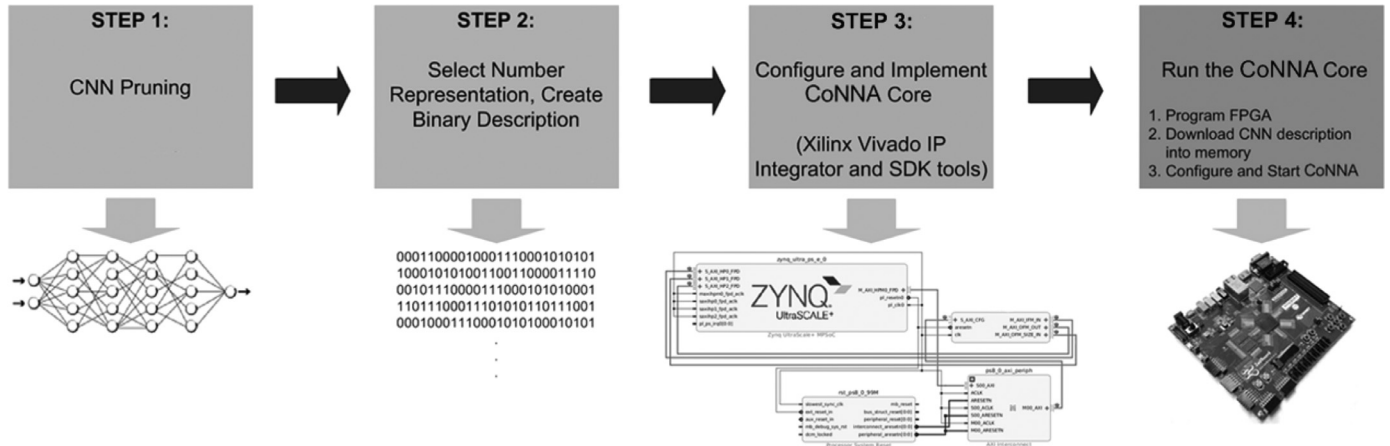


Fig. 12. Development flow for the CNN acceleration using the CoNNA CNN accelerator.

and processed by the CoNNA CNN accelerator. We have developed Python-based translator software that takes pruned CNN Keras model, performs floating-point to fixed-point conversion, determines the optimal 16-bit number format for representing all CNN related data and creates a binary version of CNN structural description linked list.

- Integrating configured instance of the CoNNA accelerator IP core using Vivado IP Integrator and modifying software application using Xilinx SDK – this step generates a complete CNN acceleration system using some of Xilinx PSoC or MPSoC family devices. Within this system specified configuration of the CoNNA CNN accelerator, with a user-defined number of PBs, will be integrated.
- Programming FPGA, downloading CNN structural description binary file into operating memory, initializing and starting the CoNNA CNN accelerator and measuring achievable CNN processing performance.

In order to perform comparison experiments, the CoNNA CNN accelerator has been configured in six different configurations, specified in Table 3. The idea was to use CoNNA configurations that are as close as possible to the configurations of the reference accelerators from Table 2, in terms of the number of computing elements (MAC units), operating frequency and number representation, to enable as fair as possible performance comparison.

All six instances of the CoNNA architecture have been implemented targeting Xilinx ZU9 MPSoC device, using the flow from

Fig. 12. Vivado Design Suite 2018.1 has been used to perform synthesis and implementation of the complete CNN acceleration system, with default Vivado synthesis and implementation settings. Implementation results for six different configurations of the CoNNA accelerator that have been used in the experiments are shown in Table 3.

From Table 3 it can be seen that all configurations have been successfully implemented using a Xilinx ZU9 MPSoC device. Please notice that the number of used DSP blocks for all configurations is larger than the number of MAC units used by the reference CNN hardware accelerators respectively, presented in Table 2. The reason for this is the fact that CoNNA uses additional DSP blocks inside the ISM module, but these additional DSP blocks are not used to perform computations defined by the target CNN. All six implementations of CoNNA architecture have been tested and benchmarked on six selected CNN architectures (AlexNet, VGG16, VGG19, GoogleNet, ResNet-18, and ResNet-50) using Xilinx ZCU102 development board. Based on these experiments performance results have been extracted and compared with seven reference CNN accelerator architectures (Eyeriss, NullHop, NVDLA, NEURAghe, CNN_A1, fpgaConvNet, and Aristotle).

4.2. Performance comparison results

The results of performance comparison experiments are presented in Table 4. For each of the reference accelerators, perfor-

Table 3
FPGA Resources required to implement CoNNA instances used in experiments.

Parameter/Configuration	CoNNA_C1	CoNNA_C2	CoNNA_C3	CoNNA_C4	CoNNA_C5	CoNNA_C6
Number of Processing Blocks	168	128	32	256	216	198
Slice LUTs	142,470	112,550	35,042	267,424	244,464	225,132
Number of BRAMs	364	284	86	596	516	480
Number of DSPs	177	137	41	277	237	219
Operating Frequency	200 MHz	60 MHz	100 MHz	140 MHz	150 MHz	214 MHz

Table 4

Performance comparison with reference CNN accelerators.

Architecture	CNN Architecture	Latency [ms/frame]	Frame Rate [frames/sec]	Performance [GOp/sec]	Effective efficiency [%]
Eyeriss ^(a)	AlexNet	115.3	34.69	46.14	68.66%
	VGG-16	4309.5	0.69	21.36	31.79%
CoNNA_C1 ^(a)	AlexNet	14.20	70.40	93.74	139.50%
	VGG-16	102.77	9.73	301.17	448.17%
NullHop	VGG-16	2269.00	0.44	13.62	88.67%
	VGG-19	2439.00	0.41	16.10	104.82%
CoNNA_C2	VGG-16	375.94	2.66	82.33	536.03%
	VGG-19	436.68	2.29	89.94	585.56%
NVDLA	AlexNet	263.01	3.80	5.51	86.09%
	GoogleNet	384.76	2.60	8.21	128.28%
	ResNet-50	1843.51	0.54	4.17	65.16%
CoNNA_C3	AlexNet	96.90	10.32	14.97	233.91%
	GoogleNet	202.02	4.95	15.63	244.18%
	ResNet-50	431.03	2.32	17.91	279.84%
NEURAghe	VGG-16	236.61	5.52	170.86	238.36%
	ResNet-18	-	6.67	23.97	33.44%
CoNNA_C4	VGG-16	127.71	7.83	242.36	338.11%
	ResNet-18	56.05	17.84	64.10	89.42%
CNN_A1	AlexNet	144.93	6.90	10.01	15.45%
	VGG-16	1639.34	0.61	18.88	29.13%
CoNNA_C5	AlexNet	21.12	47.35	68.71	106.03%
	VGG-16	145.14	6.89	213.26	329.10%
fpgaConvNet	AlexNet	52.40	19.08	27.68	65.90%
	VGG-16	633.00	1.58	55.71	132.64%
CoNNA_C1@125MHz	AlexNet	26.00	38.46	55.81	132.88%
	VGG-16	180.50	5.54	171.48	408.28%
Aristotle	VGG-16	364.96	2.74	84.81	100.08%
CoNNA_C6	VGG-16	101.83	9.82	303.96	358.68%

^a Accelerating only convolutional layers.

mance data in terms of input instance processing latency, frame rate, average compute power and efficiency are presented. The performance of each of the reference accelerators is followed by a performance of the appropriate CoNNA architecture configuration, specified in Table 3, which was configured in the identical or closest possible configuration as the reference accelerator, to allow a fair comparison. Performance data for the reference CNN hardware accelerators were taken from papers that introduced these architectures [19,23,25,29,30,32]. For the NVDLA accelerator supplied Excel sheet estimator was used to generate achievable performance data.

In the case of Eyeriss CNN accelerator, performance data is related to the processing of convolutional layers only, since Eyeriss cannot accelerate pooling and fully-connected layers. Also, input instance processing latency is reported only when Eyeriss processes input images in batches, with batch sizes of 4 and 3, for AlexNet and VGG16 CNNs respectively. Performance data for all other reference accelerators is related to processing complete CNNs, since all other architectures, as well as CoNNA, can process all layers from a CNN. Also, reported input instance processing latencies in these cases refer to the latency of processing a single input image, i.e. working with a batch size of 1.

Average computing performance for all architectures has been calculated as the total number of operations needed to process one input image by the target dense CNN network, multiplied by the time required to process one input image. Effective efficiency was calculated as the achievable average computing performance divided by the theoretical peak compute performance (calculated as the number of available MAC units multiplied by the operating frequency). Values closer to 100% mean that the CNN accelerator is more efficiently using available compute power. Values above 100% mean that the CNN accelerator is employing some optimization technique during CNN processing, effectively reducing the required number of computations.

From Table 4 it can be seen that the CoNNA architecture outperforms all reference CNN accelerator architectures in terms of

achievable frame rate and effective efficiency. When compared with MIT's Eyeriss CNN accelerator, it can be seen that the CoNNA architecture achieves higher frame rates and higher efficiency for both CNN networks used. In the case of AlexNet CNN acceleration, CoNNA is capable of reaching a 2.03 times higher frame rate than Eyeriss. In the case of VGG16 CNN acceleration, this improvement is even greater, CoNNA is able to reach a 14.10 times higher frame rate than Eyeriss. Also, it can be seen that CoNNA is able to achieve much higher effective efficiency, even above 100% due to the compressed data processing, than Eyeriss. The main reason for this improvement lies in the fact that the CoNNA architecture is able to take advantage of compressed kernel/weight and feature maps processing, due to using the "All zero-skipping" technique, and Eyeriss is not.

When compared to NullHop, CoNNA is also able to achieve better performance, but this time improvement is not as dramatic as in the case of the Eyeriss accelerator but is more consistent. CoNNA achieves 6.05 and 5.58 times faster frame rates than NullHop when processing VGG16 and VGG19 CNNs respectively. The main reason for this is that the NullHop accelerator is also being able to take advantage of the sparsity present in the input feature maps. But because NullHop is not able to take advantage of the sparsity of CNN weights, because it is not designed to process compressed CNNs, its performance is lower than CoNNA's. In the case of VGG-19 CNN, NullHop is also able to reach effective efficiency values greater than 100%, due to the fact that it is able to skip all computations where input feature map values are equal to zero.

The CoNNA architecture is also superior to NVDLA, for all three CNNs used in experiments. CoNNA is able to reach 2.72, 2.15 and 4.91 times higher frame rates than NVDLA, when accelerating AlexNet, GoogleNet and ResNet-50 CNNs respectively. However, the improvement over NVDLA is smaller than in the case of Eyeriss and NullHop, because NVDLA is using the Winograd algorithm to speed-up the convolution calculation process, which is sufficient to allow NVDLA to go above 100% of effective efficiency in case of processing GoogleNet CNN network.

Table 5
CoNNA_C3 performance breakdown for AlexNet CNN.

Layer	Active PBs	Total latency [us]	Processing latency [us]	PB efficiency [%]	Number of operations [GOp]	Performance [GOp/sec]
CONV1	32 (100%)	60,689.63	27,668.73	45.60%	0.21083	3.47
POOL1	4 (12.5%)	2274.32	1574.48	8.65%	0.00126	0.55
CONV2_1	32 (100%)	9049.99	8151.04	98.12%	0.22395	24.75
CONV2_2	32 (100%)	9049.99	8151.04	98.12%	0.22395	24.75
POOL2	4 (12.5%)	1029.36	596.72	7.24%	0.00078	0.76
CONV3	32 (100%)	5409.80	4497.29	83.13%	0.29904	55.28
CONV4_1	32 (100%)	1700.80	1342.00	78.90%	0.11214	65.93
CONV4_2	32 (100%)	1700.80	1342.00	78.90%	0.11214	65.93
CONV5_1	32 (100%)	1207.34	968.14	80.19%	0.07476	61.92
CONV5_2	32 (100%)	1207.34	968.14	80.19%	0.07476	61.92
POOL3	4 (12.5%)	138.61	46.45	4.19%	0.00016	1.15
FC6	4 (12.5%)	1943.70	1902.74	12.24%	0.07550	38.84
FC7	4 (12.5%)	886.74	845.78	11.92%	0.03355	37.83
FC8	4 (12.5%)	583.58	573.58	12.29%	0.00819	14.03

When compared with the NEURAghe CNN accelerator, CoNNA is also able to achieve better performance, being 1.42 and 2.67 times faster in terms of achievable frame rates. The NEURAghe architecture is able to achieve effective efficiency that is significantly higher than 100% in the case of processing the VGG-16 CNN network. This is curious since the authors of NEURAghe architecture didn't mention that they are using any optimization technique, like zero skipping or Winograd algorithm, during CNN processing. Furthermore, there is a significant drop in the effective computing efficiency, of around 7 times, when the ResNet-18 CNN network is accelerated using NEURAghe. This drop, although not that significant, is visible in CoNNA's performance also. This can probably be partially explained by the fact that the ResNet-18 CNN network is the network with many shallow convolutional layers, where the number of convolutional kernels is significantly lower than available 864 MAC units in NEURAghe and 256 PBs in CoNNA_C4 configuration. This means that, at least when CoNNA is concerned, there will be a significant number of idle PBs during the processing of these shallow convolutional layers, as described in Section 3.1. This idling PB inefficiency can only be partially compensated by the "All zero-skipping" technique and is the main reason why CoNNA reaches only 89.42% effective efficiency when processing the ResNet-18 CNN network.

The CoNNA architecture is clearly superior to the CNN_A1 architecture, reducing instance classification times by a factor of 6.86 and 11.30 in the case of processing AlexNet and VGG-16 CNNs respectively. A similar observation holds in the case of fpgaConvNet and Aristotle architectures. Compared with the fpgaConvNet architecture CoNNA architecture is able to reach frame rates that are 2.02 and 3.08 times higher. Similar to the NEURAghe architecture, in the case of fpgaConvNet architecture, there is a significant difference in the effective efficiency when processing AlexNet and VGG-16 CNN networks. Furthermore, the efficiency of the fpgaConvNet architecture when processing VGG-16 CNN is higher than 100%, indicating that the fpgaConvNet architecture must be using some optimization technique to speedup CNN processing, although authors of fpgaConvNet didn't comment on this in the original paper. Finally, in the case of the Aristotle architecture, this improvement is slightly better, CoNNA is reaching 3.58 times faster frame rates when processing the VGG-16 CNN network, compared with Aristotle's performance.

4.3. CoNNA CNN computational performance analysis

In order to better understand how the CoNNA architecture processes CNNs, and where the bottlenecks are Table 5 presents detailed execution information for the CoNNA_C3 configuration when it is accelerating AlexNet CNN.

Table 5 presents the following data for each AlexNet CNN layer. The "Active PB" field holds the number of Processing Blocks, out of 32 available, which are actually being active during the processing of the current CNN layer. The "Total Latency" stands for the total time needed to process a CNN layer, including the time required to preload CNN coefficients/weights, perform required computations, and store the final results. The "Processing Latency" is the time that is actually spent performing all computations defined in the current CNN layer, excluding any preparatory steps and is therefore always shorter than the "Total Latency" time. The "PB Efficiency" value is calculated as the ratio of the "Processing Latency" and the "Total Latency", multiplied with the percentage of active PBs for the current CNN layer. The "Number of Operations" is the total number of arithmetic operations required to be performed in order to process the current CNN layer. The "Performance" is the computational throughput for the current CNN layer, calculated as the ratio of the "Number of Operations" and the "Total Latency".

From Table 5 it can be seen that the CoNNA architecture is being able to maintain high utilization of PBs over all convolutional layers, irrelevant of their size and kernel characteristics. The efficiency of processing convolutional layers is extremely important since they are the most computationally demanding layer type in modern CNNs. It can also be seen that, as we move to deeper convolutional layers of the CNN, PB efficiency starts to drop. This is because more time has to be spent in preloading kernel coefficient values into PBs before actual computation can start. The reason for this is that the depths of convolutional layers increase as we go deeper in the CNN, resulting in more kernel weights data that has to be preloaded into PBs. Since CoNNA uses limited bandwidth data bus to transfer data to the PBs, it takes increasingly more time to preload convolutional kernel data as the size of convolutional kernels increases.

From Table 5 it can also be seen that the PB efficiency drops significantly when CoNNA is processing pooling and fully-connected layers. Processing of these layer types is data movement intensive, so available data bandwidth of CoNNA's data bus is again the main limiting factor. However, these layers are not computationally intensive, as can clearly be seen from Table 5, so this inefficiency is not significantly degrading the total performance of the CoNNA architecture. From Table 5 it can be seen that the number of operations for all pooling and fully-connected layers in the case of AlexNet CNN constitutes only 8% of the total number of computations, but it takes CoNNA only 9.45% of total CNN compute time to compute these layers. This means that there is an increase of only 1.45% in total CNN compute time despite very low PB efficiency values attained during pooling and fully-connected layer processing.

Finally, Fig. 13 presents data throughput waveforms measured on CoNNA's Input Stream and Output Stream interfaces, in the

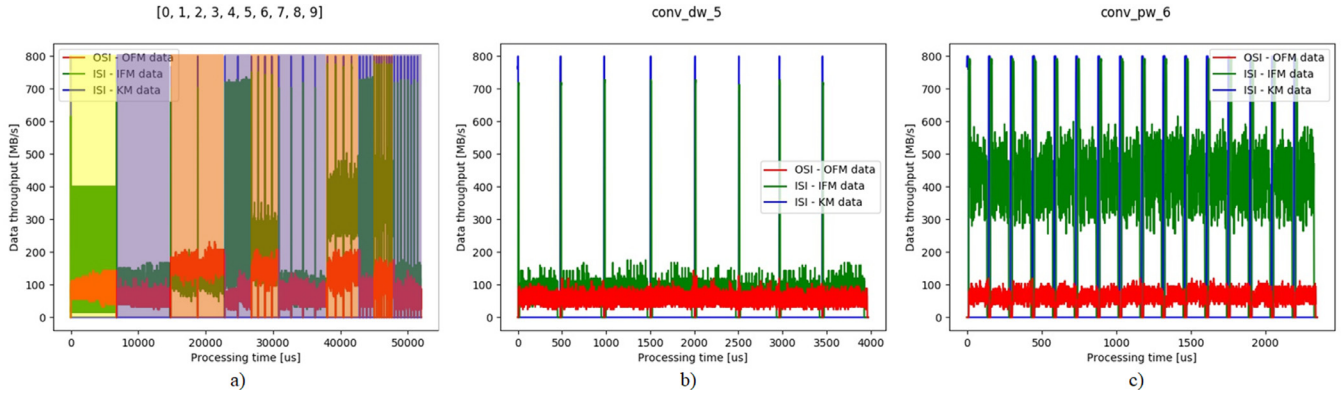


Fig. 13. Data throughput waveforms over CaNNA's ISI and OSI interfaces in case of MobileNet V1 CNN processing using CoNNA_C3 Instance: a) During the processing of first 10 MobileNet V1 CNN Layers; b) During processing 5th Depthwise Convolutional Layer; c) During the processing of 6th Pointwise Convolutional Layer.

case when MobileNet V1 CNN is being accelerated. The CoNNA configuration that was used in this experiment was CoNNA_C3 from Table 3. Please notice that each graph actually presents data throughputs on both ISI and OSI interfaces. Redline shows data throughput over the OSI interface, which is actually the data throughput of writing output feature map data to the operating memory. Blue and green lines together represent data throughput over the ISI interface. Blue line shows the data throughput during the loading of convolutional kernel coefficients, which precedes every convolutional kernel group calculation phase. Finally, the green line shows data throughput during the IFM data processing phase, during which IFM data is loaded into ISB memory from the Input Stream Manager module.

In the Fig. 13a data throughput on CoNNA's ISI and OSI interfaces during the processing of the first 10 layers from MobileNet V1 CNN is shown. The reason why data throughput during complete MobileNet V1 CNN processing was not shown is that processing times of latter layers from MobileNet V1 are so short, compared to processing times of initial layers, that nothing could be seen on the graph. From Fig. 13a processing periods of different MobileNet V1 layers are clearly visible. Processing starts with the processing of the first convolutional layer, which uses 3×3 kernels to transform the input image, designated by the yellow segment. After this layer is completed first depthwise convolutional layer, designated by the purple segment, is processed next. Next comes the first pointwise convolutional layer, designated by the orange segment in the Fig. 13a, which is actually a standard convolutional layer that uses 1×1 kernels to process the input feature map. After this layer, a succession of depthwise and pointwise convolutional layer pairs follows, which is clearly visible by alternating purple and orange segments in the Fig. 13a. Because dataflows required for processing convolutional layer with 3×3 kernels, depthwise convolutional layer with 3×3 kernels and pointwise convolutional layer with 1×1 kernels differ, data throughputs over ISI and OSI interfaces during the processing of these layer types differ also, which can clearly be visible on Fig. 13a.

From Fig. 13a it can be seen that data throughput over the OSI interface is relatively constant during CNN processing, without sudden bursts. In contrast, data throughput over the ISI interface has a more irregular waveform, with different patterns and bursts of activity, depending on the layer type that is being processed by the CoNNA accelerator. It can be seen that there are periodic peaks of high throughput values, which actually correspond with periods of convolutional kernel loading into Local Memory modules within PBs and initial filling of the ISB memory with IFM data. During the actual convolution calculation process throughput over the ISI interface is significantly lower, because of high IFM data reuse by adjacent convolutions, as explained in Section 3.2.2.

Fig. 13b presents data throughput waveforms during the processing of the 5th depthwise convolutional layer from MobileNet V1 CNN. Analyzing the ISI interface throughput waveform it can be observed that it is composed out of eight almost identical segments. The reason why there are exactly eight almost identical segments is related to the characteristic of the 5th depthwise convolutional layer and used configuration of the CoNNA accelerator in the following way. The number of different convolutional kernels used within the 5th depthwise convolutional layer from MobileNet V1 CNN equals $Kernel_Num = 256$. On the other hand, the CoNNA_C3 configuration uses 32 Processing Blocks, i.e. $Num_PB = 32$. This means that the loop L3 from Algorithm 3 dis-

Algorithm 3

Pseudo-code of convolutional layer "All zero-skipping" processing algorithm, implemented in the CoNNA architecture.

```

L1: for (x = 0; x < OFM_Width; x++)
L2:   for (y = 0; y < OFM_Height; y++)
L3:     for (kn = 0; kn < Kernel_Num; kn += Num_PB)
W1:       while (PBs_Busy())
         {OFM[x][y][kn] += Calc_Next_NZPT(CIFM[x*Sh:x*Sh+Kernel_Width-
           1][y*Sv:y*Sv+Kernel_Height-1][:]);
          CKM[kn][:][:][:];
          OFM[x][y][kn+1] +=
            Calc_Next_NZPT(CIFM[x*Sh:x*Sh+Kernel_Width-
              1][y*Sv:y*Sv+Kernel_Height-1][:]);
            CKM[kn+1][:][:][:];...
          OFM[x][y][kn+Num_PB-1] +=
            Calc_Next_NZPT(CIFM[x*Sh:x*Sh+Kernel_Width-
              1][y*Sv:y*Sv+Kernel_Height-1][:]);
            CKM[kn+Num_PB-1][:][:][:];}
```

cussed in Section 3.1 will, in this case, be unrolled with the partial unrolling factor of 32. This means that 32 different depthwise convolutions, from a total of 256, will be calculated in parallel, by available PBs. In order to calculate all 256 different depthwise convolutions, the same process has to be repeated 8 times, each time using a different group of 32 depthwise convolutions. At the beginning of the processing of each group of convolutions, corresponding convolutional kernels need to be loaded into Local Memory modules of each PB. During this short period of time, there is a sudden rise in the data throughput over the ISI interface, shown by the blue line in the Fig. 13b. After kernel preloading is completed, the processing of IFM data can commence. At the beginning IFM data is fetched at a high rate, indicated by sudden peaks in the green line, immediately following the peaks on the blue line in the Fig. 13b, because ISB cache memory inside the Input Stream Interface module is initially empty and is being filled up by the Write Controller module. Once IFM data reuse becomes active, IFM data throughput falls sharply, because the WC module now has to

wait for RCU to consume individual IFM sticks from the ISB cache before replacing them, which can also be visible by inspecting the green line from the Fig. 13b. Since the 5th depthwise convolutional layer uses 3×3 convolutional kernels, a theoretical decrease in the data throughput should be around nine times compared with the throughput during the initial filling of the ISB cache memory. However, since CoNNA is processing compressed data and performs zero-skipping, actual data throughput is higher and more irregular, since compression ratios and distributions of zeros vary from one IFM stick to another.

Finally, Fig. 13c presents data throughput waveforms during the processing of a different CNN layer type, this time it is the 6th pointwise convolutional layer from MobileNet V1 CNN. In this case, there are 16 almost identical segments, because this layer contains 512 different convolutional kernels. What can also be seen from Fig. 13c is that the required throughput over the ISI interface during the actual convolution calculation process is now significantly higher compared with one from Fig. 13b. The reason for this is that in this case, CoNNA is actually computing 1×1 convolutions, so there is no IFM stick reuse.

4.4. Analysis of the impact of non-zero distribution of the sparse weights and activations on the CoNNA architecture CNN processing efficiency

Since CoNNA operates directly on sparse kernel and input feature maps during CNN processing, and since non-zero values distribution within the kernel and IF maps, in general, don't follow any regular pattern, it would be interesting to investigate how this affects CoNNA's CNN processing efficiency. When the CoNNA architecture is concerned, irregular distribution of non-zero values within the kernel and IF maps can potentially affect CoNNA's CNN processing efficiency in two following ways:

1. Irregular non-zero values distribution within the kernel and IF maps can lead to a different number of non-zero product terms that have to be computed when the same IFM bundle is being convolved with different convolutional kernels. Since CoNNA has to wait until all *Processing Blocks* complete their convolution computation process before proceeding with the computation of the next group of convolutions, some PBs will be idling, waiting for the slowest one to finish its work, and the actual convolutional layer processing time would be prolonged. This issue was discussed in Section 3.1 when Algorithm 3 for CNN processing, which is actually being implemented by the CoNNA architecture, was presented.
2. Irregular non-zero values distribution within the kernel and IF maps can lead to a situation where there could actually be no non-zero product terms in the current non-zero product term search window, as the Data Fetcher module searches for the next non-zero product term to forward to the Computing Unit. This situation will result in one additional idle clock cycle for the Computing Unit during product term computation and will prolong the time required to compute one convolution operation.

Let us first analyze the severity of the non-equal convolution computation times of different PBs on the CoNNA's CNN processing efficiency. Since CoNNA computes Num_PB different convolutions in parallel, Num_PB being the number of Processing Blocks that are available within the current CoNNA instance, if different PBs have to compute a different number of non-zero product terms, this can lead to different convolution computation times for different PBs. In this scenario, some of PBs could be idling, waiting for the other PBs to complete their convolution computation process, before proceeding with the next convolution computation process

using the next IFM bundle. In theory, the expected number of non-zero product terms required to compute one convolution is directly dependant on the percentage of non-zero values within the current kernel and IFM bundle, as the following equation shows

$$Num_MACs_Ideal = Kernel_Width \cdot Kernel_Height \cdot Kernel_Depth \cdot P_IFMNZ \cdot P_KMNZ \quad (5)$$

where P_IFMNZ is the percentage of non-zero valued points in the input feature map bundle, and P_KMNZ is the percentage of non-zero valued convolutional coefficients in the convolutional kernel. In this ideal scenario, each convolution computation operation would require identical time to complete

$$Convolution_Compute_Time_Ideal = Num_MACs_Ideal \cdot T_{clk} \quad (6)$$

where T_{clk} is the period of the clock signal used to synchronize CoNNA's operation.

However, due to the irregularity of non-zero values distribution within each kernel and IFM bundle, the actual number of non-zero product terms can and will probably vary for each kernel/IFM bundle pair, leading to a different convolution compute time for each active PB. Since CoNNA must wait until all active PBs complete their convolution computation process before proceeding with the next group of convolutions, the actual compute time for each group of Num_PB convolutions will be longer than the ideal convolution computation time, as the following equation shows

$$\begin{aligned} Convolution_Compute_Time_Actual &= \max_{i \in Active_PBs} \{Num_MACs_Actual_i \cdot T_{clk}\} \\ &\geq Convolution_Compute_Time_Ideal \end{aligned} \quad (7)$$

In order to investigate how severe this increase in compute time could be, a series of experiments have been performed, using convolutional kernels of different sizes, with varying percentages of non-zero values present within the kernel and IFM bundle. The results of these experiments are shown in the Fig. 14. Each of the graphs, shown in the Fig. 14, shows how the convolution computation time changes, depending on the actual percentage of non-zero values present in the kernel and IFM bundle, for $3 \times 3 \times 32$, $3 \times 3 \times 64$, $3 \times 3 \times 128$, $3 \times 3 \times 256$, $3 \times 3 \times 512$, and $3 \times 3 \times 1024$ convolution kernels. In all these experiments percentage of non-zero values that were present in the kernel and IFM bundle has been varied independently, within the [10% - 100%] range.

What can be observed in Fig. 14 is that there is indeed an increase in the actual convolution computation time, compared to the ideal theoretical estimate. This computational time increase is more severe, as the percentage of non-zero values present in the kernel and IFM bundle decreases. However, this increase in the computational time is gradual for high and moderate percentages of the non-zero kernel and IFM bundle values and rises sharply only when the percentage of the non-zero kernel and IFM bundle values approaches relatively small values, less than 30%. Since the achievable percentages of non-zero values after CNN pruning are rarely lower than 30% [19,43], this means that the increase in the convolution computation time will be acceptable, being less than 10%, for most of current CNN architectures. Furthermore, from Fig. 14 it can also be observed that the maximum increase in the convolution computation time decreases, as larger convolutional kernels are being used. For example, maximum increase in the convolution computation time for $3 \times 3 \times 32$ kernel can be as high as 54%, but as we move to larger kernels it quickly decreases to 36%, 22%, 17%, 11%, and 8%, in case of $3 \times 3 \times 64$, $3 \times 3 \times 128$, $3 \times 3 \times 256$, $3 \times 3 \times 512$, and $3 \times 3 \times 1024$ convolutional kernels respectively. Most of state-of-the-art CNN architectures dominantly use deep convolutional layers, having only several relatively

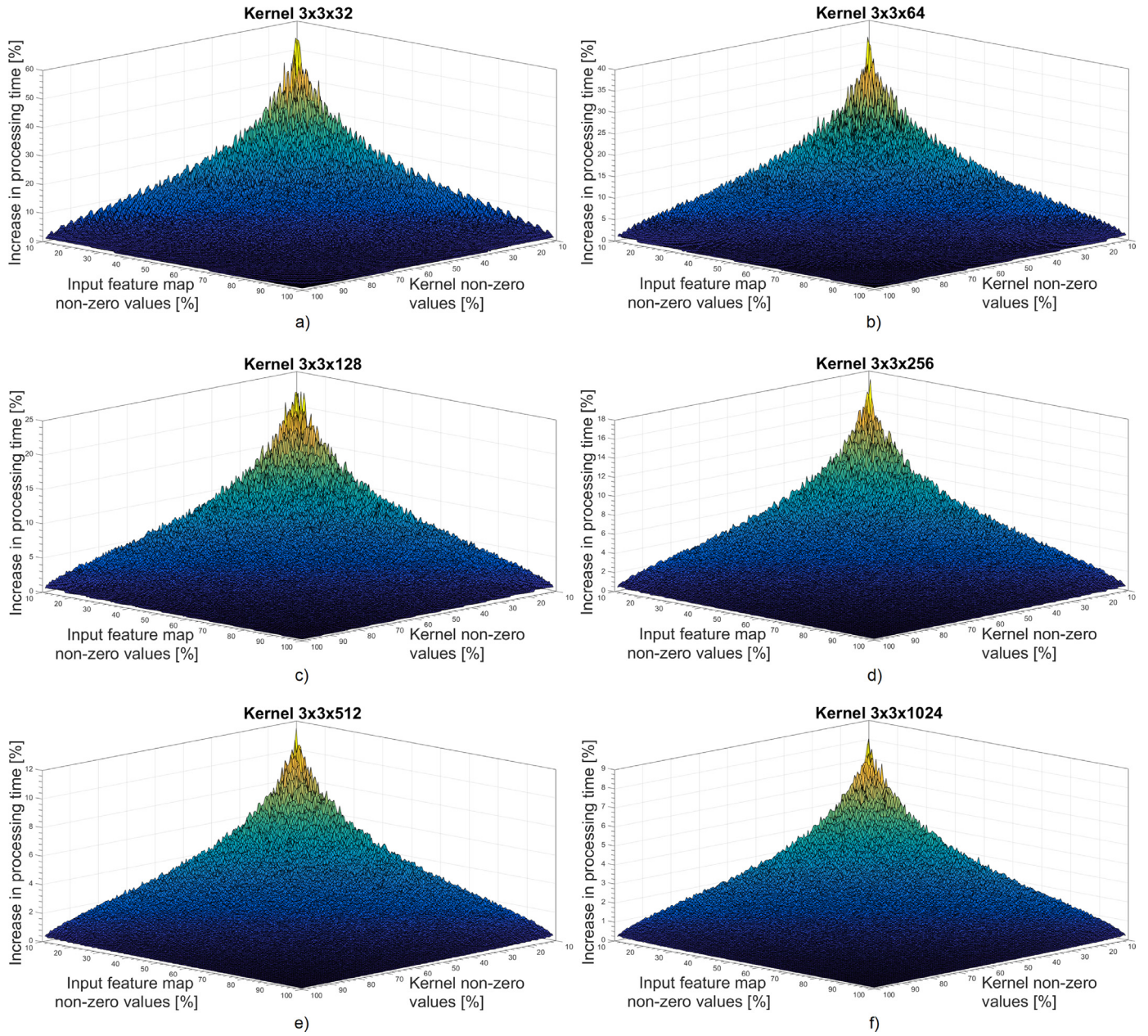


Fig. 14. Relative increase in convolution layer processing time due to irregular distribution of non-zero kernel and IFM bundle values for: a) $3 \times 3 \times 32$ kernel; a) $3 \times 3 \times 32$ kernel; b) $3 \times 3 \times 64$ kernel; c) $3 \times 3 \times 128$ kernel; d) $3 \times 3 \times 256$ kernel; e) $3 \times 3 \times 512$ kernel; f) $3 \times 3 \times 1024$ kernel.

shallow layers at the beginning of the CNN network. This is a very favorable situation for the CoNNA architecture, meaning that the relative increase in processing time will usually be well below 10% from ideal, theoretical processing time, for most current CNN architectures. At the end of this section, a detailed analysis of the actual increase in the CNN layer processing time due to irregular distribution of non-zero kernel and IFM bundle values for two standard CNN networks, AlexNet and VGG-16, will be presented in order to further justify this conclusion.

Next, let us analyze the impact of the irregular distribution of non-zero kernel and IFM bundle values on the efficiency of non-zero product term detection logic, located within the Data Fetcher module, as shown in Figs. 8 and 9. As explained in Section 3.2.1, the Data Fetcher module searches for the non-zero product terms and forwards them to the Computing Unit which performs the actual convolution computation process. To make this non-zero prod-

uct term search effective, the Data Fetcher module performs a parallel search, checking a number of candidate product terms concurrently, in what is called a "Search Window". This is necessary in order to keep the Computing Unit busy since if the non-zero product term detection process would have been implemented sequentially, the result would be an extremely inefficient convolution calculation process, where the Computing Unit would be idle most of the time, effectively canceling any processing speedup due to zero skipping. The question is how to select the optimal size of this "Search Window", in order to minimize the number of CU idle clock cycles, while keeping required logic resources for DF module implementation as low as possible. Intuitively, as this "Search Window" becomes larger, the probability of not detecting at least one non-zero product term within it would decrease. In order to investigate how the probability of having "Empty Search Windows" depends on the "Search Window" size and the probability of non-

Average Percentage of "Empty Search Windows" vs "Window Size" - no Internal FIFO

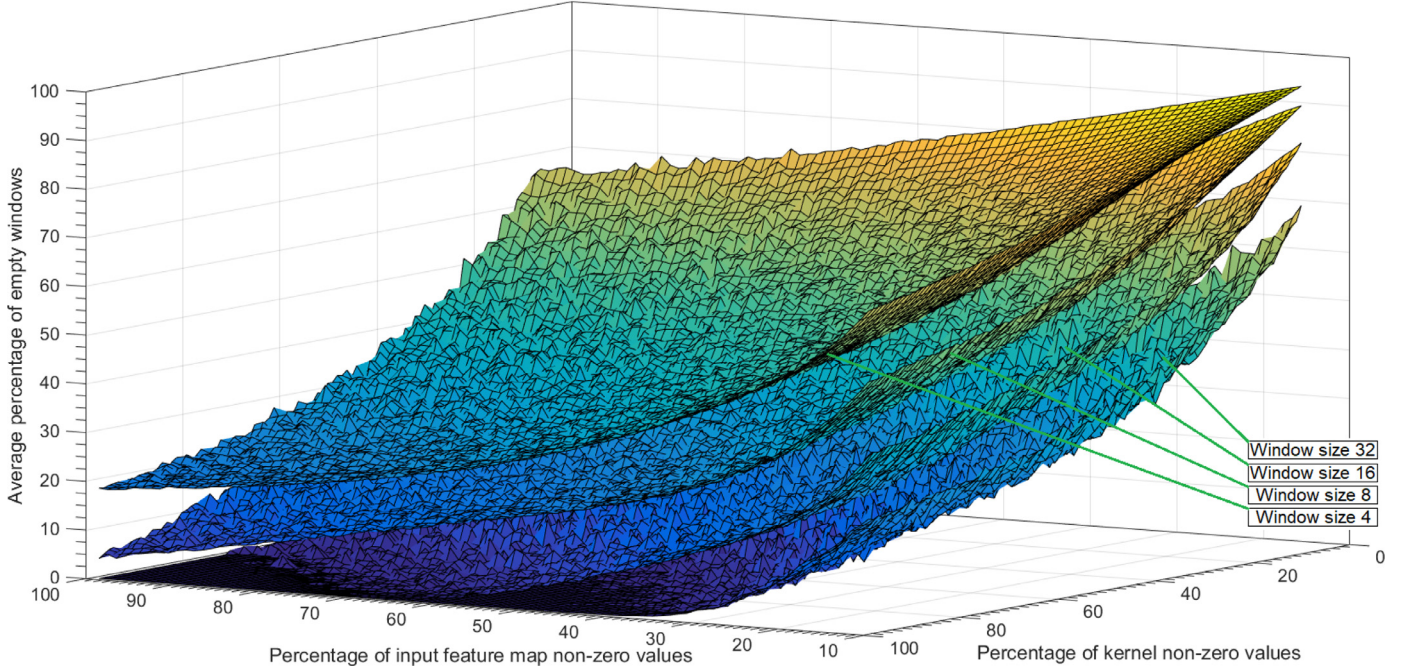


Fig. 15. Average percentage of "Empty Search Windows" as the function of percentage of non-zero values present in kernel and IFM bundle in the case of data fetcher module with no built-in FIFO, for four different sizes of "Search Window" parameter.

Average Percentage of "Empty Search Windows" vs "Window Size" - Ideal Internal FIFO

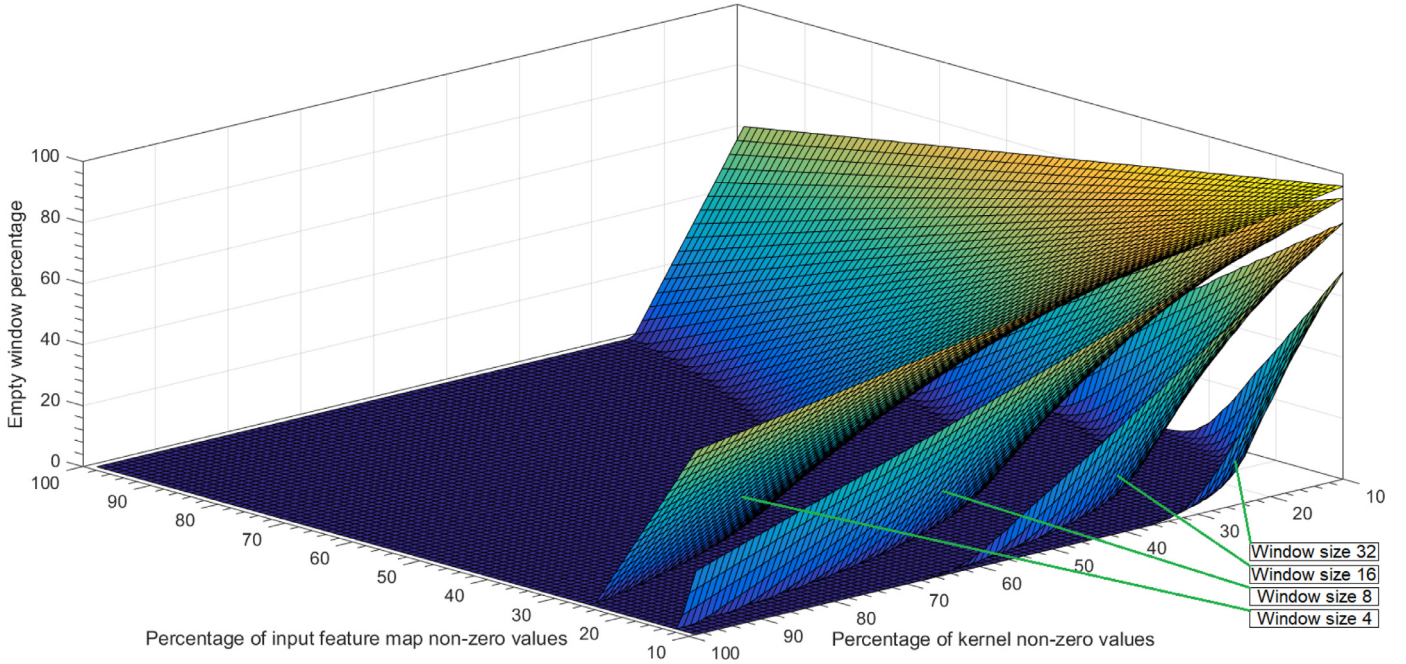


Fig. 16. Average percentage of "Empty Search Windows" as the function of percentage of non-zero values present in kernel and IFM bundle in the case of data fetcher module with built-in ideal FIFO, for four different sizes of "Search Window" parameter.

zero kernel and IFM bundle values, a number of experiments have been performed, and the results are shown in Figs. 15–17.

Fig. 15 shows the average percentage of "Empty Search Windows" as a function of the percentage of kernel and IFM bundle non-zero values, for four sizes of the "Search Window", 4, 8, 16 and 32 product terms wide. From Fig. 15 it can be seen that the size of "Search Window" plays a significant role in the average percentage of "Empty Search Windows". For the "Search Window" size of 4, even when kernel and IFM bundle contain a very large number of non-zero valued elements, more than 90%, there will be a signifi-

cant percentage of "Empty Search Windows", at least 20%, during the computation of individual convolutions. As we increase the size of "Search Window", the percentage of "Empty Search Windows" starts to fall and in the case of the "Search Window" size of 32, it stays below 10% even when there are only 50% of non-zero valued elements in the kernel and IFM bundle. On the other hand, when there is a small percentage of non-zero values in kernel and IFM bundle, less than 20%, the percentage of "Empty Search Windows" becomes very high for all considered "Search Window" sizes, going even above 90% in the case of "Search Window" size of 4.

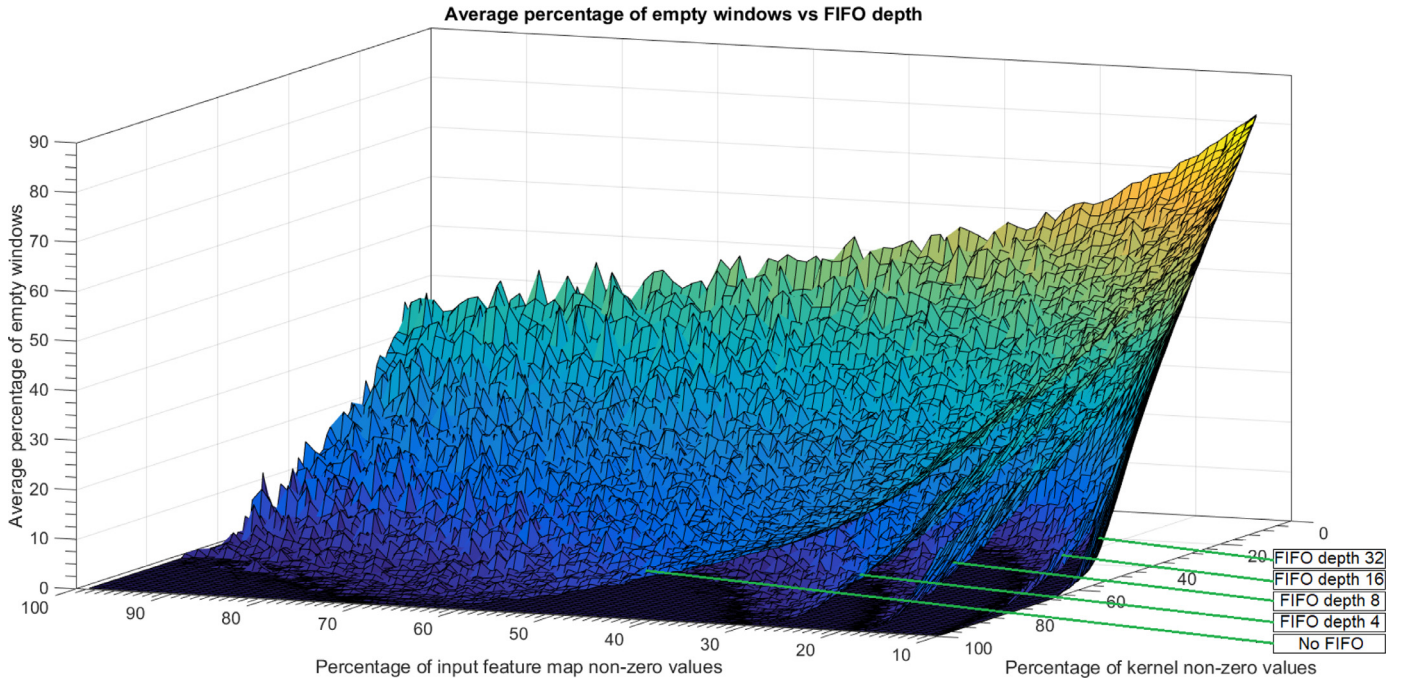


Fig. 17. Average percentage of “Empty Search Windows” as the function of percentage of non-zero values present in kernel and IFM bundle in the case of data fetcher module with “Search Window” size of 16, with built-in FIFO of depth 0, 4, 8, 16 and 32.

In the experiments, resulting in the average percentage of “Empty Search Windows” surfaces, shown in the Fig. 15, Data Fetcher module was operating without a built-in FIFO, which results in the simplest possible design. However, because of the irregular distribution of non-zero kernel and IFM bundle values, the actual number of detected non-zero product terms in each “Search Window” will probably vary, so there will be situations wherein the current “Search Window” there is more than one non-zero product term detected, but in the subsequent “Search Windows” there are no non-zero product terms. In this scenario, a simple Data Fetcher module without a FIFO will not be able to avoid idle clock cycles, since it cannot start searching for non-zero product terms in the next “Search Window” until all non-zero product terms, detected in the current “Search Window” are computed. Using Data Fetcher with an internal FIFO, this situation could be avoided. It would be interesting to analyze how the size of this FIFO impacts the average percentage of “Empty Search Windows”, and can a large enough FIFO completely eliminate “Empty Search Windows”. In order to investigate this, a new set of experiments has been conducted, this time with the Data Fetcher module that uses an infinitely large FIFO, and the results of these experiments are shown in the Fig. 16.

As can be seen from Fig. 16, even using an infinitely deep FIFO cannot remove all “Empty Search Windows”. This was to be expected since when the average expected number of non-zero product terms falls below the value of one per current “Search Window” size, even ideal FIFO cannot eliminate all “Empty Search Windows”. This is clearly visible in the Fig. 16. When the following inequality is satisfied

$$\text{“Search Window Size”} < \frac{1}{P_{IFMNZ} \cdot P_{KMNZ}} \quad (8)$$

non-zero product terms will be, on average, separated by more than “Search Window Size” zero product terms and even ideal FIFO will not be able to eliminate all “Empty Search Windows”. For example, in the case of “Search Window” size of 4, the average percentage of “Empty Search Windows” surface intersects with the zero “Empty Window Percentage” plane at the circle defined by

the following equation

$$P_{IFMNZ} \cdot P_{KMNZ} = \frac{1}{4} \quad (9)$$

which is clearly visible in the Fig. 16. However, the benefit of using a FIFO inside Data Fetcher is clearly visible, since it can significantly reduce the average percentage of “Empty Search Windows” for all “Search Window” sizes, up to a theoretical limit, as explained above.

What can also be observed from Fig. 16 is as we increase the “Search Window” size, the intersection curve between the “Empty Search Windows” surface and the zero “Empty Window Percentage” plane moves steadily to the right corner. If we could use the “Search Window” of infinite size we could completely eliminate all “Empty Search Windows”, but of course this is not possible in practice.

In the experiments presented in the Fig. 16, an infinitely deep FIFO was assumed, which cannot be used in practice. Therefore, it would be interesting to analyze how a more realistic FIFO of finite depth, reduces the percentage of “Empty Search Windows”. Fig. 17 presents the average percentage of “Empty Search Windows” in the case when the Data Fetcher module with the “Search Window” size of 16 is used, for five different depths of built-in FIFO, 0, 4, 8, 16 and 32.

From Fig. 17 it can be seen as the depth of used FIFO increases, the average percentage of the “Empty Search Windows” surface approaches the ideal average percentage of the “Empty Search Windows” surface, shown in the Fig. 16. However, from Fig. 17 it can be clearly seen that using even a FIFO of depth 32 is enough to reach the ideal average percentage of the “Empty Search Windows” surface. Because of this, in the current version of the CoNNA architecture, Data Fetcher modules were configured to use “Search Window” of size 16, with a built-in FIFO of depth 32.

What Figs. 14 and 17 show is that when there is a high percentage of zeros present in the kernel and/or input feature map, a significant degradation of CoNNA’s performance could be present, either because of non-balanced number of non-zero product terms that are present in the convolutional operations that are being

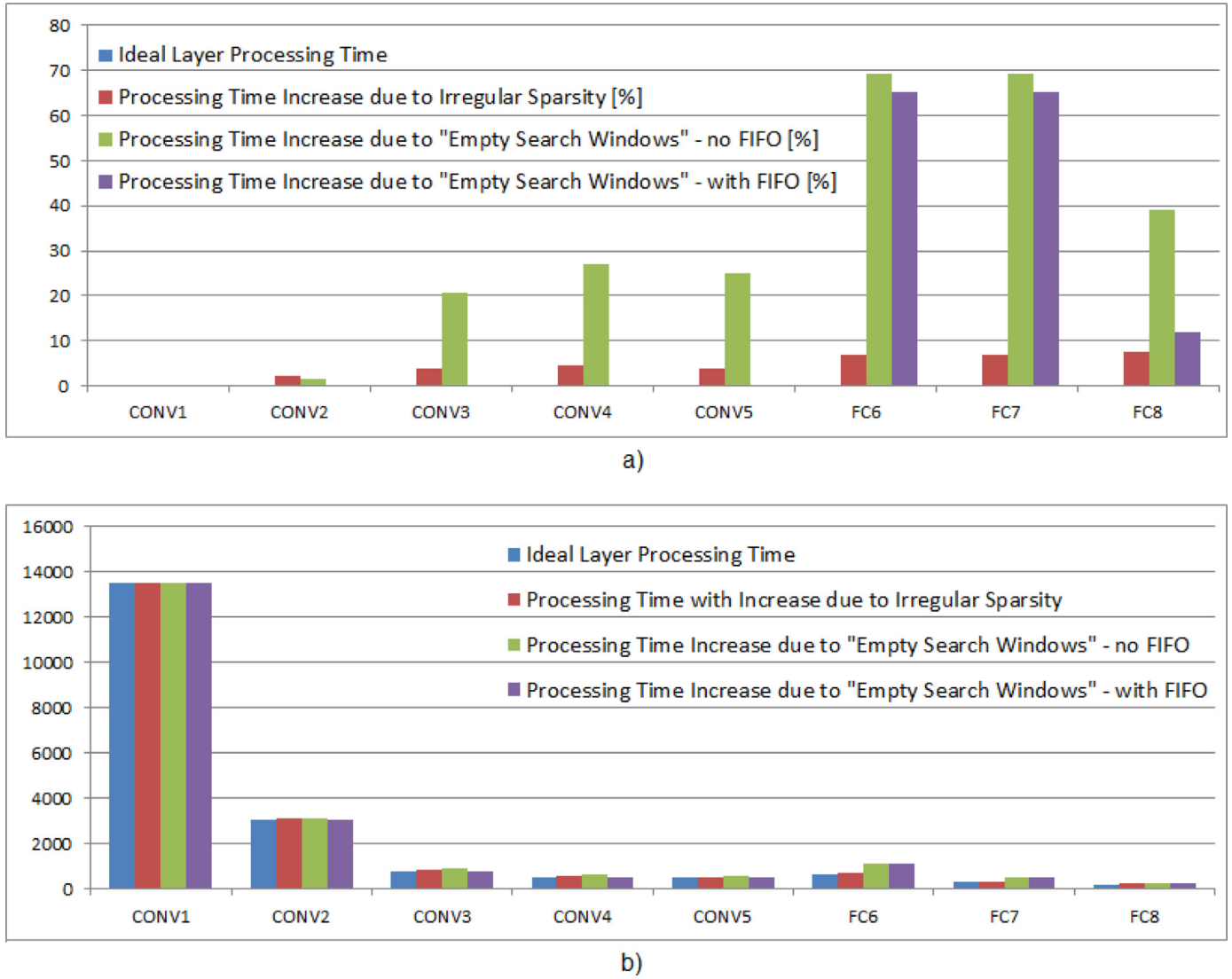


Fig. 18. Increase in layer compute times for AlexNet CNN, due to various effects of irregular distribution of non-zero values in kernel and input feature maps: a) Percentage increase in individual layer compute time; b) Absolute layer processing time.

computed in parallel by available PBs, or because of “Empty Search Windows” that are present during the non-zero product term search process. However, when working with realistic CNN architectures, the situation where there is a high percentage of zeros present in the kernel and/or input feature map usually happens only within the layers that are located deep inside the CNN, and mostly within the fully-connected layers. Please notice that compute times of these layers contribute only slightly to the total compute time of complete CNN. Therefore, any increase in the compute times of deep convolutional or fully-connected layers due to irregular non-zero values distribution within kernel and input feature maps of these layers should result in only a small increase in the total CNN compute time.

In order to analyze the magnitude of the increase of total CNN compute time, due to irregular non-zero values distribution within kernel and input feature maps, in the case of realistic CNN networks, AlexNet and VGG-16 CNN networks have been used. For both of these CNNs, individual layer compute times have been computed, for several scenarios:

1. ideal layer compute time when the increase in the layer compute time due to the irregular non-zero values distribution is not taken into account,
2. layer compute time when the non-balanced number of non-zero product terms that are present in the convolutional operations that are being computed in parallel by available PBs is taken into account,
3. layer compute time when the increase in compute time due to “Empty Search Windows” is taken into account, when the Data Fetcher module uses a 16 product terms wide “Search Window” but is not using a built-in FIFO, and
4. layer compute time when the increase in compute time due to “Empty Search Windows” is taken into account, when the Data Fetcher module uses a 16 product terms wide “Search Window” and a 32-deep built-in FIFO is being used.

Figs. 18 and 19 present the results of these experiments for AlexNet and VGG-16 CNN networks respectively. In both Figures, two graphs are presented. First, showing the percentage increase in the individual layer compute times, and the second, showing the absolute layer compute times.

From Fig. 18 and 19 it can be seen that the layer processing time increase due to a non-balanced number of non-zero product terms that are present in the convolutional operations is not significant. For two selected CNN networks, when convolutional layers are considered, the maximum processing time increase is less than

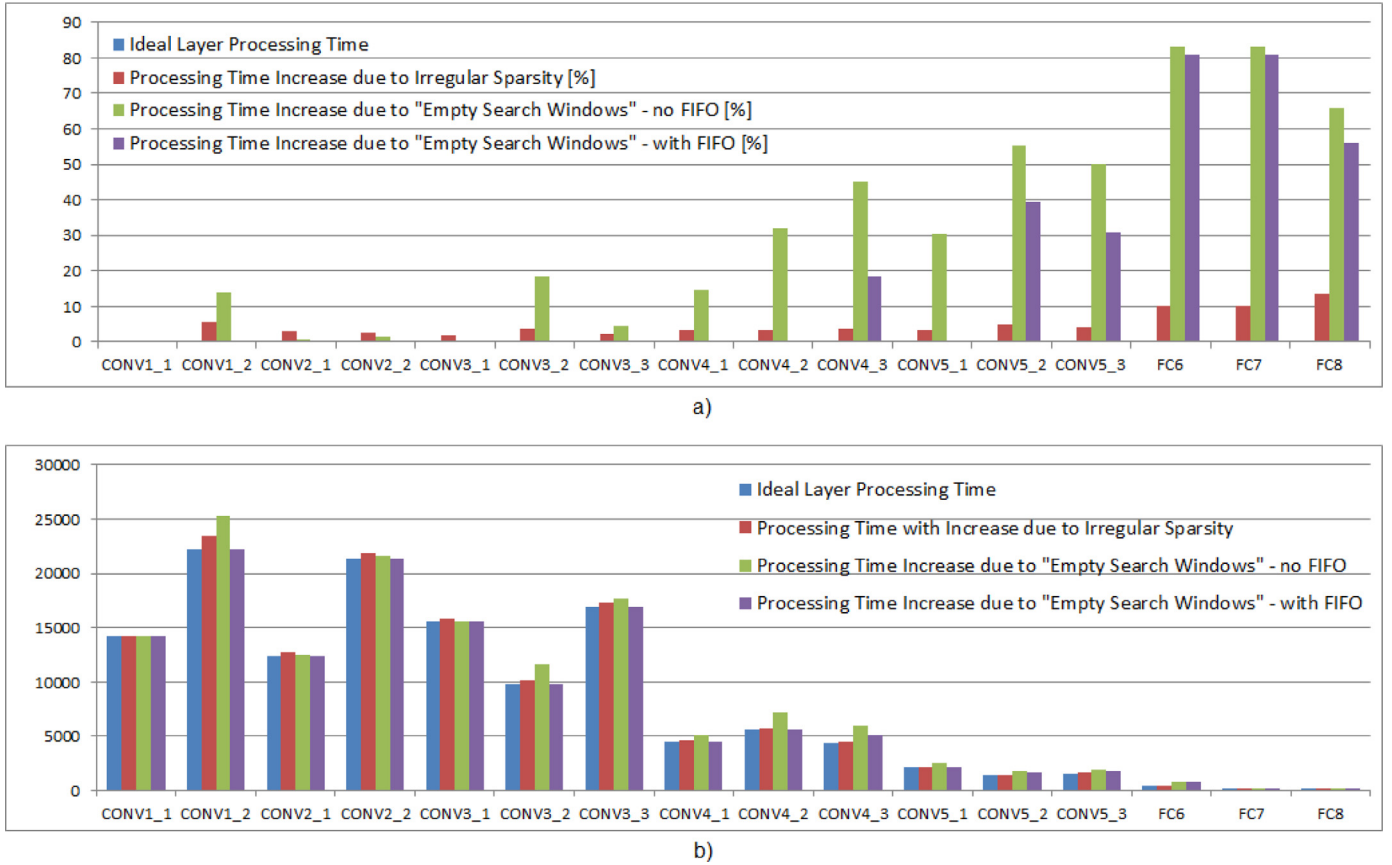


Fig. 19. Increase in layer compute times for AlexNet CNN, due to various effects of irregular distribution of non-zero values in kernel and input feature maps: a) Percentage increase in individual layer compute time; b) Absolute layer processing time.

5% compared to ideal compute time. When fully-connected layers are concerned, this increase is slightly higher and can reach as high as 13%, as is the case with the FC8 VGG-16 layer. However, as can be observed in Figs. 18b and 19b, total compute times of fully-connected layers for both AlexNet and VGG-16 CNNs are almost negligible in comparison with convolutional layer compute times, so this increase in fully-connected layer processing times doesn't significantly change the total CNN processing time. In fact, the total CNN processing time increase due to a non-balanced number of non-zero product terms in the case of AlexNet and VGG-16 CNNs is only 0.95% and 2.57% respectively.

When the individual layer processing times increase due to the "Empty Search Windows" is considered, from Figs. 18 and 19 it can be observed that it is more severe, and can reach the values as high as 80%, for FC6 and FC7 VGG-16 layers. Also, it can be seen that this increase is also significant for the convolutional layers which are located deeper within the CNN network, and can reach the values of 40%. However, once more, the contribution of all these layers to the total CNN processing time, especially the contribution of fully-connected layers, is small in comparison with the contribution of bigger convolutional layers located closer to the beginning of the CNN network. This is visible when the increase in the total CNN processing time, due to "Empty Search Windows" for AlexNet and VGG-16 CNNs is computed. In the case of AlexNet total processing time increase is 2.94%, and for the VGG-16 network, it is even smaller, only 1.10%.

From Figs. 18 and 19, the benefit of using a built-in FIFO within Data Fetcher modules can also be seen. When FIFO is being used, the resulting increase in layer processing times can be completely

eliminated, or reduced, for almost all convolutional layers, and reduced for all fully-connected layers.

Based on all experiments that were performed in order to analyze the effects of irregular non-zero values distribution within kernel and input feature maps on the CoNNA architecture CNN processing time, it can be concluded that the irregular non-zero values distribution has indeed an effect on the increase of CoNNA's total CNN processing time, but that this increase is relatively small, not exceeding 3% of ideal CNN processing time.

5. Conclusion

In this paper, a novel CNN hardware accelerator, CoNNA, has been proposed. CoNNA is a coarse-grained reconfigurable hardware architecture capable of accelerating complete pruned and compressed CNNs, employing the "All zero-skipping" technique to decrease required CNN processing time by skipping all ineffectual operations during convolutional, pooling and fully-connected layer processing. It can be used to accelerate convolutional, depthwise convolutional, pooling, fully-connected, concatenation and adding layers of the target CNN network. CoNNA is designed to process compressed CNNs, as well as input feature maps, which seems to enable achieving higher processing performance values when compared to some of the previously proposed CNN accelerator solutions. CoNNA is designed to act as a co-processor soft-IP core that is connected to a host processor within contemporary SoC or PSoC architectures. The CoNNA architecture has been implemented using Xilinx ZynqUltrasc+ FPGA family and compared with seven previously proposed hardware CNN accelerators. The CoNNA archi-

ture, when configured to use identical number of MAC units and operating at the same clock frequency as previously proposed MIT's Eyeriss, NullHop and NVIDIA's NVDLA, NEURAghe, CNN_A1, fpgaConvNet, and DeepPhi's Aristotle CNN accelerators, enables up to 14.10, 6.05, 4.91, 2.67, 11.30, 3.08 and 3.58 times faster CNN execution of standard CNN networks respectively.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444.
- [2] K. Fukushima, Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position, *Biol. Cybern.* 36 (4) (1980) 193–202.
- [3] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, in: *Advances in Neural Information Processing Systems*, Lake Tahoe, 2012, pp. 1097–1105.
- [4] R. Girshick, R.-C.N.N. Fast, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition - CVPR*, 15, 2015, pp. 1440–1448.
- [5] Jonathan Long, Evan Shelhamer, Trevor Darrell, Fully convolutional networks for semantic segmentation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition - CVPR*, 15, 2015, pp. 3431–3440.
- [6] L. Deng, J. Li, J.-T. Huang, K. Yao, D. Yu, F. Seide, M. Seltzer, G. Zweig, X. He, J. Williams, Y. Gong, Recent advances in deep learning for speech research at microsoft, in: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Vancouver, 2013, pp. 8604–8608.
- [7] C. Chen, A. Seff, A. Kornhauser, J. Xiao, Deepdriving: learning affordance for direct perception in autonomous driving, in: *Proceedings of the IEEE International Conference on Computer Vision*, Santiago, 2015, pp. 2722–2730.
- [8] A. Esteva, B. Kuprel, R.A. Novoa, J. Ko, S.M. Swetter, H.M. Blau, S. Thrun, Dermatologist-level classification of skin cancer with deep neural networks, *Nature* 542 (7639) (2017) 115–118.
- [9] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis, Mastering the game of Go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484–489.
- [10] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, *arXiv preprint*, arXiv:1409.1556, 2014.
- [11] NVIDIA Volta Architecture Whitepaper WP-08608-001_v1.1, NVIDIA, (2018), [Online], Available: <http://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>.
- [12] NVIDIA Jetson TX2 Delivers Twice the Intelligence to the Edge, NVIDIA, (2017), [Online], Available: <https://devblogs.nvidia.com/jetson-tx2-delivers-twice-intelligence-edge/>.
- [13] Intel® Stratix® 10 Variable Precision DSP Blocks User Guide, Intel FPGA Group, (2017), [Online], Available: <https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/hb/stratix-10/ug-s10-dsp.pdf>.
- [14] E. Nurvitadhi, S. Subhaschandra, G. Boudoukh, G. Venkatesh, J. Sim, D. Marr, R. Huang, J. OngGeeHock, Y.T. Liew, K. Srivatsan, D. Moss, Can FPGAs beat GPUs in accelerating next-generation deep neural networks? in: *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays - FPGA*, 17, Monterey, 2017, pp. 5–14.
- [15] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, O. Temam, DaDianNao: a machine-learning supercomputer, in: *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, Cambridge, 2014, pp. 609–622.
- [16] K. Guo, S. Lingzhi, J. Qiu, S. Yao, S. Han, Y. Wang, and H. Yang, Angel-eye: a complete design flow for mapping cnn onto customized hardware, in: *2016 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Pittsburgh (2016), 24–29.
- [17] X. Wei, C.H. Yu, P. Zhang, Y. Chen, Y. Wang, H. Hu, Y. Liang, J. Cong, Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs, in: *Proceedings of the 54th Annual Design Automation Conference*, Austin, 2017, p. 29.
- [18] C. Wang, L. Gong, Q. Yu, X. Li, Y. Xie, X. Zhou, DLAU: a scalable deep learning accelerator unit on FPGA, *IEEE Trans. Comput.-Aid. Des. Integr. Circuits Syst.* 36 (3) (2017) 513–517.
- [19] Y.H. Chen, T. Krishna, J.S. Emer, V. Sze, Eyeriss: an energy-efficient reconfigurable accelerator for deep convolutional neural networks, *IEEE J. Solid-State Circuits* 52 (1) (2017) 127–138.
- [20] N. Suda, V. Chandra, G. Dasika, A. Mohanty, Y. Ma, S. Vrudhula, J.S. Seo, Y. Cao, Throughput-optimized OpenCL-based FPGA accelerator for large-scale convolutional neural networks, in: *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Monterey, 2016, pp. 16–25.
- [21] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, Going deeper with embedded fpga platform for convolutional neural network, in: *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Monterey, 2016, pp. 26–35.
- [22] Z. Liu, Y. Dou, J. Jiang, J. Xu, S. Li, Y. Zhou, Y. Xu, Throughput-Optimized FPGA accelerator for deep convolutional neural networks, *ACM Trans. Reconfigurable Technol. Syst.* 10 (3) (2017) 17.
- [23] P. Meloni, A. Capotondi, G. Deriu, M. Brian, F. Conti, D. Rossi, L. Raffo, L. Benini, NEURAghe: exploiting CPU-FPGA synergies for efficient and flexible CNN inference acceleration on Zynq SoCs, *ACM Trans. Reconfigurable Technol. Syst.* 11 (3) (2018) 18.
- [24] N. Shah, P. Chaudhari, K. Varghese, Runtime programmable and memory bandwidth optimized FPGA-Based coprocessor for deep convolutional neural network, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (12) (2018) 5922–5934.
- [25] S.I. Venieris, C.S. Bouganis, fpgaConvNet: mapping regular and irregular convolutional neural networks on FPGAs, *IEEE Trans. Neural Netw. Learn. Syst.* (2018) 1–17 Early Access.
- [26] J. Cheng, J. Wu, C. Leng, Y. Wang, Q. Hu, Quantized CNN: a unified approach to accelerate and compress convolutional networks, *IEEE Trans Neural Netw Learn Syst* 29 (10) (2017) 4730–4743.
- [27] M. Motamedi, P. Gysel, S. Ghiasi, PLACID: a platform for FPGA-based accelerator creation for DCNNs, *ACM Trans. Multimed. Comput. Commun. Appl.* 13 (4) (2017) 62 Article No.
- [28] Y. Choi, D. Bae, J. Sim, S. Choi, M. Kim, L.S. Kim, Energy-Efficient design of processing element for convolutional neural network, *IEEE Trans. Circuit. Syst. II* 64 (11) (2017) 1332–1336.
- [29] X. Chen, Z. Yu, A flexible and energy-efficient convolutional neural network acceleration with dedicated ISA and accelerator, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 26 (7) (2018) 1408–1412.
- [30] K. Guo, S. Han, S. Yao, Y. Wang, Y. Xie, H. Yang, Software-Hardware code-sign for efficient neural network acceleration, *IEEE Micro.* 37 (2) (2017) 18–25.
- [31] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N.E. Jerger, A. Moshovos, Cnvlutin: ineffectual-neuron-free deep neural network computing, in: *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, Seoul, 2016, pp. 1–13.
- [32] A. Aïmar, H. Mostafa, E. Calabrese, A. Rios-Navarro, R. Tapiador-Morales, I.A. Lungu, M.B. Milde, F. Corradi, A. Linares-Barranco, S.C. Liu, T. Delbruck, NullHop: a flexible convolutional neural network accelerator based on sparse representations of feature maps, *IEEE Trans. Neural Netw. Learn. Syst.* (2018) 1–13 Early Access.
- [33] Y. Lu, C. Wang, L. Gong, X. Zhou, SparseNN: a performance-efficient accelerator for large-scale sparse neural networks, *Int. J. Parallel Program.* 46 (4) (2018) 648–659.
- [34] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, Y. Chen, Cambricon-x: an accelerator for sparse neural networks, in: *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, Taipei, 2016, p. 20. Article No.
- [35] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M.A. Horowitz, W.J. Dally, EIE: efficient inference engine on compressed deep neural network, in: *Proceedings of the 43rd International Symposium on Computer Architecture*, Seoul, 2016, pp. 243–254.
- [36] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, W.J. Dally, Scnn: an accelerator for compressed-sparse convolutional neural networks, in: *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, Toronto, 2017, pp. 27–40.
- [37] S. Anwar, W. Sung, Compact Deep Convolutional Neural Networks With Coarse Pruning, *ArXiv preprint*, arXiv:1610.09639, (2016).
- [38] H. Li, A. Kadav, I. Durdanovic, H. Samet, H.P. Graf, Pruning filters for efficient convnets, *ArXiv preprint*, arXiv:1608.08710, (2016).
- [39] J.H. Luo, J. Wu, W. Lin, Thinet: a filter level pruning method for deep neural network compression, in: *The IEEE International Conference on Computer Vision (ICCV'17)*, Venice, 2017, pp. 5058–5066.
- [40] Y. He, X. Zhang, J. Sun, Channel pruning for accelerating very deep neural networks, in: *International Conference on Computer Vision (ICCV'17)*, Venice, 2017, pp. 1389–1397.
- [41] S. Anwar, K. Hwang, W. Sung, Structured pruning of deep convolutional neural networks, *ACM J. Emerg. Technol. Comput. Syst.* 13 (3) (2017) 32 Article No.
- [42] S. Han, J. Pool, J. Tran, W. Dally, Learning both weights and connections for efficient neural network, in: *Advances in neural information processing systems (NIPS 2015)*, Montreal, 2015, pp. 1135–1143.
- [43] H. Song, H. Mao, W.J. Dally, Deep compression: Compressing deep Neural Networks With pruning, Trained Quantization and Huffman Coding, *arXiv preprint*, arXiv:1510.00149, (2015).
- [44] Y. Guo, A. Yao, Y. Chen, Dynamic network surgery for efficient dnns, in: *Advances In Neural Information Processing Systems*, Barcelona, 2016, pp. 1379–1387.
- [45] A. Erdeljan, B. Vukobratović, R. Struharik, IP core for efficient zero-run length compression of CNN feature maps, in: *25th Telecommunications Forum (TELFOR 2017)*, Belgrade, 2017, pp. 44–49.
- [46] M. Horowitz, Computing's energy problem (and what we can do about it), in: *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC'14)*, San Francisco, 2014, pp. 10–14.
- [47] A. Zhu, T. Wang, H. Snoussi, Hierarchical graphical-based human pose estimation via local multi-resolution convolutional neural network, *AIP Adv.* 8 (3) (2018) 1–14.

- [48] M. Al Rahhal, Y. Bazi, T. Abdullah, M. Mekhalfi, H. AlHichri, M. Zuair, Learning a multi-branch neural network from multiple sources for knowledge adaptation in remote sensing imagery, *Remote Sens.* 10 (12) (2018) 1–18.
- [49] T. Chen, S. Lu, J. Fan, SS-HCNN: semi-Supervised hierarchical convolutional neural network for image classification, in: *IEEE Transactions on Image Processing*, Early Access, 2018, pp. 1–10.
- [50] NVIDIA Deep Learning Accelerator, [Online], Available: <http://nvdla.org/>, (2019).
- [51] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, Boston, 2015, pp. 1–9.
- [52] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, Las Vegas, 2016, pp. 770–778.